

Mosquitto MQTT — Operator Manual

What this technology does

Mosquitto is used at two separate boundaries:

1. **Gateway-01 local broker** — loopback-only persistent store-and-forward buffer between MQTT Forwarder and the cloud.
2. **Cloud brokers on ULC-01/ULC-02** — preferred/backup MQTT service with separate gateway, ChirpStack, Node-RED, and evidence identities/listeners.

The cloud pair is **not** a replicated live-session cluster. Do not restore round-robin session routing.

Current port ownership

Gateway-01 local Mosquitto:	127.0.0.1:1883
Public gateway MQTT via HAProxy:	:8883 on ingress candidate
Cloud Mosquitto gateway-facing TLS backend:	:8884
Cloud Mosquitto ChirpStack workload TLS:	:8885
Cloud Mosquitto Node-RED mTLS:	:8886
HAProxy ChirpStack MQTT service:	:18883
HAProxy Node-RED MQTT service:	:18884

The exact bind address matters. A listener on the wrong interface can be both a security and routing defect.

Part A — Gateway local broker

Step A1 — Verify process and loopback listener

Run on Gateway-01:

```
ps w | grep '[m]osquitto'
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

PASS means: Mosquitto is running and 1883 is bound to 127.0.0.1, not 0.0.0.0, LAN, or WAN.

Step A2 — Verify persistence/queue limits without exposing secrets

```
grep -E
'^ (persistence|persistence_location|persistence_file|autosave_interval|max_queued_messages|max_queued_bytes|l
istener|protocol|allow_anonymous|include_dir)' /etc/mosquitto/mosquitto.conf
```

Expected commissioned pattern includes persistence, finite message/byte limits, and a loopback listener. Do not make an unlimited queue on an SD card.

Step A3 — Verify cloud bridge metadata

```
grep -E '^(connection |address
|remote_clientid|cleansession|bridge_cafile|bridge_certfile|bridge_keyfile|bridge_insecure|topic )'
/etc/mosquitto/conf.d/bridge.conf
```

PASS means:

- bridge target is the commissioned cloud MQTT service on 8883;
- CA/cert/key paths are configured;
- bridge_insecure is false;
- uplink/event/state traffic uses the intended outgoing QoS/session behavior;
- command/downlink traffic is inbound under the exact gateway EUI hierarchy.

Never cat the private key.

Step A4 — Verify MQTT Forwarder publishes locally

In one gateway shell:

```
mosquitto_sub -h 127.0.0.1 -p 1883 -t 'as923/gateway/0016c001f139a1cb/event/#' -v
```

Generate one real approved sensor uplink.

PASS means: a message arrives under the correct AS923/EUI topic.

If nothing arrives, the failure is before the WAN bridge. Diagnose RAK5146/Concentrator/MQTT Forwarder/local listener first.

Step A5 — Verify the bridge is connected

```
logread | grep -Ei 'mosquitto|bridge|connected|connack|tls|certificate' | tail -n 100
ss -tnp 2>/dev/null | grep ':8883' || netstat -tnp 2>/dev/null | grep ':8883' || true
```

PASS means: the bridge maintains an authenticated TLS/MQTT session without rapid reconnect/authentication failure.

Part B — Cloud brokers

Step B1 — Discover the actual service name on each broker host

Run on ULC-01 and ULC-02:

```
for s in mosquitto mosquitto-1 mosquitto-2 mosquitto-3; do
  sudo systemctl is-active "$s" >/dev/null 2>&1 && echo "$s"
done
```

Use the service name returned by the host. Do not assume an old unit name.

Step B2 — Check listeners

```
sudo ss -lntp | grep -E ':(8884|8885|8886)\b' || true
```

Compare the actual bind addresses with the cloud runbook. A missing listener is not fixed by restarting ChirpStack.

Step B3 — Check bounded logs

Replace <SERVICE> with the service name discovered in B1:

```
sudo journalctl -u <SERVICE> --since=-15min --no-pager | tail -n 120
```

Look for client-certificate, username/ACL, bridge, listener, or socket errors. Do not use an unbounded journal dump during routine diagnosis.

Step B4 — Verify gateway mTLS identity behavior

The commissioned gateway certificate identity is the exact EUI 0016c001f139a1cb, and the gateway ACL is limited to its own as923/gateway/0016c001f139a1cb/... hierarchy.

Use the protected test/commissioning certificate path defined by the MQTT runbook. A correct test proves:

1. trusted gateway certificate can connect;
2. it can publish only its allowed event/state hierarchy;
3. it can subscribe only to its command hierarchy;
4. a client with no certificate is rejected;
5. unrelated topic access is rejected.

Do not loosen ACLs just to make a test utility connect.

Step B5 — Verify consumers separately from publishers

When application data is missing, prove both sides:

```
publisher reached broker?  
subscriber connected/subscribed?  
message on exact expected topic?  
consumer processed that message?
```

A quiet container stdout log does not prove MQTT traffic is absent. Subscribe with an authorized diagnostic identity or inspect broker/session evidence.

Preferred/backup rule

The cloud brokers provide service failover, not session replication. Normal topology is preferred Mosquitto plus backup. If the preferred broker fails, clients reconnect through the approved HA path; when it recovers, do not create rapid round-robin switching.

Safe change procedure

1. identify the exact listener/workload affected;
2. copy the current configuration and ACL to protected rollback storage;
3. validate file ownership/permissions;
4. change one broker first when possible;
5. validate Mosquitto configuration before service restart using the method supported by the installed package;
6. restart/reload only the broker being changed;
7. verify listeners and intended client identity;
8. verify unauthorized/no-cert rejection;
9. verify real gateway/ChirpStack/Node-RED flow;
10. only then apply the same change to the second broker.

Do not simultaneously restart both cloud brokers for a normal configuration edit.

Common failure map

Symptom	First check
Local gateway publish fails	Gateway local broker/Forwarder
Local works, bridge disconnected	LTE/DNS/TLS/client cert/cloud 8883 path
TLS succeeds, MQTT authorization fails	CN/username/ACL/topic
Cloud receives gateway event, ChirpStack does not	ChirpStack workload listener/routing/subscription
Node-RED misses events but ChirpStack works	Node-RED 18884/8886 path and subscription
Duplicate app rows after reconnect	downstream idempotency, not weaker MQTT delivery

Backup/recovery rules

- Gateway `mosquitto.db` is an availability queue, not immutable evidence.
- Never delete it during a transient outage merely to clear an error.
- Preserve broker config, ACLs, CA/cert/key paths, and persistence state before replacement.
- If restoring cloud brokers, restore intended preferred/backup topology; do not claim broker-session replication.

Completion checklist

Gateway side:

- local broker running;
- `127.0.0.1:1883` only;
- persistence/finite limits present;
- real local event observed;
- TLS bridge established.

Cloud side:

- expected broker services active;
- correct workload listeners bound;
- gateway mTLS/ACL behavior correct;
- preferred/backup behavior preserved;
- one real event reaches the intended consumer.

Detailed runbooks

- [../gateway/setup/04-configure-local-mqtt-buffer.md](#)
- [../gateway/setup/05-configure-mqtt-forwarder.md](#)
- [../server/cloud-production/08-mqtt-and-valkey.md](#)
- [../server/cloud-production/14-observability-alerting-and-logging.md](#)
- [../gateway/operations/05-troubleshooting.md](#)

Supporting runbook: 04-configure-local-mqtt-buffer.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Gateway 4. Configure the Persistent Local MQTT Buffer

This procedure installs a local Mosquitto broker on ChirpStack Gateway OS Base and configures it as a persistent, bounded store-and-forward queue for offline uplink buffering.

Gateway Architecture & Complete End-to-End Data Path



Detailed Layer Breakdown:

- 1. **Physical Radio Layer (RAK5146-115 HAT):** Captures incoming LoRa radio signals from sensors on AS923 frequencies (923.2–924.6 MHz) and streams raw packets to the Raspberry Pi over the SPI bus interface

(/dev/spidev0.0).

2. **Radio Control Daemon (ConcentratorD)**: Interfaces directly with the SX1303 baseband chip, handles GPIO reset control, decodes raw radio frames into structured data, and serves them on a local IPC socket (`ipc:///tmp/concentratorD_event`).
3. **Protocol Converter Daemon (MQTT Forwarder)**: Reads frames from ConcentratorD and converts them into standardized ChirpStack Gateway MQTT payload messages. It publishes these messages with **QoS 1** (At-Least-Once delivery) exclusively to the local loopback broker (`tcp://127.0.0.1:1883`).
4. **Local Persistent Buffer (On-Device Mosquitto Broker)**: Acts as a bounded store-and-forward queue. The configuration in this manual stores persistent broker state at `/etc/mosquitto/data/mosquitto.db`. During a WAN outage, QoS 1 uplinks can accumulate until the configured message/byte limit or available storage is reached. This is an availability control, not an immutable evidence store.
5. **Mutual TLS Bridge (WAN Transport)**: The local Mosquitto broker maintains outbound mTLS bridges to your remote cloud/edge server (`<MQTT_BROKER_FQDN>:8883`). When connectivity is active, or returns after an outage, it securely streams gateway event/state traffic under `as923/gateway/<GATEWAY_EUI>/...`; the separate downlink bridge receives the gateway command hierarchy.

Security boundary: what this buffer proves

The gateway must be treated as a **transport and availability component**, not as the authority that proves a sensor reading is genuine.

```
LoRaWAN sensor
-> LoRaWAN frame with MIC and frame counter
-> RAK5146 / Gateway OS
-> local Mosquitto availability buffer
-> mTLS
-> remote MQTT
-> ChirpStack validates LoRaWAN protocol/session state
-> application telemetry
-> TimescaleDB + Fabric outbox
-> Fabric adapter creates canonical evidence and digest
-> Hyperledger Fabric
```

The controls in this gateway manual protect different risks:

- loopback-only MQTT prevents ordinary LAN/WAN clients from writing directly to the local broker;
- Linux ownership and permissions restrict ordinary processes from reading or changing the queue and private key;
- finite queue limits prevent an outage from filling the SD card;
- mTLS protects and authenticates the gateway-to-cloud transport;
- LoRaWAN MIC/frame-counter validation occurs at ChirpStack after the gateway forwards the frame.

They **do not make** `mosquitto.db` **cryptographically tamper-proof against a root or physical attacker**. A privileged attacker can potentially alter, truncate, or delete files on the gateway. Do not create the authoritative Hyperledger digest on the Raspberry Pi and do not claim that a SHA-256 stored beside `mosquitto.db` would solve this problem; an attacker with the same write access could replace both the data and the local hash.

The gateway baseline now adds a separate **software integrity journal** beside ConcentratorD. That journal does not alter `mosquitto.db`; it records an independent sequence of raw gateway observations, hash-chains records and bounded segments, and periodically sends the current chain state to a server checkpoint service. After an outage the server verifies segment continuity and compares the journal with an independently captured remote gateway MQTT event before higher-level evidence can be promoted.

This materially strengthens tamper detection without additional gateway hardware, but it does not make a fully disconnected, root-compromised Raspberry Pi cryptographically unforgeable. A privileged attacker who controls

the whole Pi throughout the unanchored offline interval can potentially rebuild a different internally consistent tail. Hardware-backed monotonic state or source-level device signatures remain future stronger mitigations for that residual threat.

The application-side Fabric manuals preserve `telemetry-attestation-v1` for historical behavior and define a gateway-verified `telemetry-attestation-v2` path. For v2, the canonical evidence is sealed through OpenBao only after the server gateway-evidence verifier reports verified. The evidence private key is never stored on this gateway.

[!IMPORTANT] The completion condition for **this** manual is secure bounded buffering and transport. Complete [Gateway 4A](#) separately for tamper-evident gateway history. Even with both controls, a full root compromise remains a security incident and the software-only unanchored offline tail is not claimed to be immutable.

Before you begin

Complete these procedures first:

- [Configure Concentrator](#)
- [Secure the remote MQTT broker](#)
- [Provision the gateway MQTT identity](#)

Have these confirmed values and files ready:

```
Gateway address: <GATEWAY_IP>
Gateway EUI: <GATEWAY_EUI>
Region topic prefix: as923
Remote broker: <MQTT_BROKER_FQDN>:8883
Broker CA certificate: <MQTT_CA_CERT_FILE>
Gateway certificate: <GATEWAY_MQTT_CERT_FILE>
Gateway private key: <GATEWAY_MQTT_KEY_FILE>
```

Step 1: Back up Gateway OS

What this step does

Generates and downloads a baseline system backup archive (`.tar.gz`) from LuCI (**System > Backup / Flash Firmware**) to your administration workstation.

Why we do it

Provides a clean rollback point before installing new software packages (`opkg`) or modifying OpenWrt configuration files.

Procedure

In LuCI:

1. Open **System > Backup / Flash Firmware**.
2. Select **Generate archive**.
3. Save the archive safely on your workstation outside the gateway.

Step 2: Check package feed and storage availability

What this step does

Updates OpenWrt package feeds (`opkg update`), verifies `mosquitto-ssl` availability, and checks overlay filesystem storage space (`df -h /`).

Why we do it

Ensures that storage space is sufficient on the microSD card overlay partition and that the official OpenWrt package feed has `mosquitto-ssl` available for installation before attempting package setup.

Procedure

Run over SSH:

```
opkg update
opkg list | grep -E '^mosquitto-(ssl|client-ssl) '
opkg print-architecture
df -h /
mount
```

Continue only when: - `mosquitto-ssl` and `mosquitto-client-ssl` are available in the package list. - The writable overlay filesystem (/) has at least 50–100 MB of free storage space.

Step 3: Choose finite queue limits

What this step does

Calculates upper limits for disk space (`<BUFFER_MAX_BYTES>`), message count (`<BUFFER_MAX_MESSAGES>`), and state auto-save intervals (`<BUFFER_AUTOSAVE_SECONDS>`).

Why we do it

Prevents unlimited disk usage from filling the SD card overlay partition during extended internet outages, which would corrupt the operating system filesystem.

Procedure

Estimate your required queue size:

```
required queue bytes = peak uplinks per hour
                      × maximum outage hours
                      × measured MQTT bytes per uplink
                      × safety factor (1.5)
```

Recommended production values for Raspberry Pi 4B gateway:

Parameter	Recommended Value	Engineering & Mathematical Rationale
<BUFFER_MAX_MESSAGES>	100000	Covers ~16–24 hrs of multi-device outages. LoRaWAN MQTT payloads average ~1 KB. For 50 sensors transmitting every 30s (~6,000 uplinks/hr), 100,000 messages buffers ~16.6 hours of zero-loss traffic while capping RAM memory allocation (~50–80 MB) to protect against OOM crashes.
<BUFFER_MAX_BYTES>	104857600 (100 MB)	SD Card Protection. Calculated as $\$100,000 \text{ msgs} \times \sim 1,000 \text{ bytes/msg} = 104,857,600 \text{ bytes}$. Capping disk buffer at 100 MB prevents Mosquitto from filling 100% of the SD card root partition during multi-day network failures, which would brick the OS.
<BUFFER_AUTOSAVE_SECONDS>	60	Flash Wear vs. Data Loss Balance. Flash memory (SD cards) has limited write cycles. Writing every 1s causes rapid SD card degradation. Writing every 60s reduces SD card write operations by 60x while limiting data loss to a maximum of 60 seconds if power is abruptly cut.
<BUFFER_FREE_SPACE_RESERVE_BYTES>	52428800 (50 MB)	Emergency System Reserve. Guarantees that system services (syslog, DHCP leases, OpenWrt config commits, SSH sessions) always have 50 MB of guaranteed free disk space on / to prevent OS system panics.

Step 4: Install Mosquitto

What this step does

Installs mosquitto-ssl and mosquitto-client-ssl via opkg, verifies the mosquitto system user account, and configures OpenWrt UCI to use static configuration files (/etc/mosquitto/mosquitto.conf).

Why we do it

Installs the local broker software with SSL/TLS bridge capabilities and ensures OpenWrt launches Mosquitto using your static custom configuration file instead of dynamically generated UCI defaults.

Procedure

Run over SSH:

```
opkg install mosquitto-ssl mosquitto-client-ssl
opkg list-installed | grep '^mosquitto'
id mosquitto
```

Confirm that the mosquitto user account was created.

Configure OpenWrt UCI to use static file configuration:

```
cp /etc/config/mosquitto /etc/config/mosquitto.before-buffer 2>/dev/null || true
```

Edit /etc/config/mosquitto:

```
vi /etc/config/mosquitto
```

Ensure the configuration contains:

```
config owrt 'owrt'
    option use_uci '0'
```

Step 5: Create persistent storage directories

What this step does

Creates /etc/mosquitto/data (for persistent queue database storage) and /etc/mosquitto/certs (for TLS certificates) and sets strict file permissions (chmod 0750).

Why we do it

Isolates queue storage and cryptographic certificates on persistent disk sectors with restricted user permissions so private keys and buffered messages are protected from unauthorized access.

Procedure

Run over SSH:

```
mkdir -p /etc/mosquitto/data /etc/mosquitto/certs
chown mosquitto:mosquitto /etc/mosquitto/data
chmod 0750 /etc/mosquitto/data
chown root:mosquitto /etc/mosquitto/certs
chmod 0750 /etc/mosquitto/certs

df -h /etc/mosquitto/data
mount | grep -E 'overlay| / '
```

Verify that /etc/mosquitto/data resides on persistent overlay storage (not /tmp RAM disk).

Step 6: Copy and verify certificate files

What this step does

Installs only the known-good CA plus exact-EUI client certificate/key, cleans debris from failed setups, applies gateway-side ownership, validates certificate contents, and proves the certificate/key pair matches.

Procedure

1. Clean a reused gateway before importing credentials

```
/etc/init.d/mosquitto stop
mv /etc/mosquitto/certs /etc/mosquitto/certs.before-fix 2>/dev/null || true
```

```
mkdir -p /etc/mosquitto/certs
chown mosquitto:mosquitto /etc/mosquitto/certs
chmod 0750 /etc/mosquitto/certs
```

A zero-byte `ca.crt`, a certificate for an old EUI, or generic credentials from an earlier attempt can make TLS troubleshooting misleading. Keep old files outside the active directory.

2. Copy only the current gateway files

On the administration workstation:

```
scp <MQTT_CA_CERT_FILE> root@<GATEWAY_IP>:/tmp/ca.crt
scp <GATEWAY_MQTT_CERT_FILE> root@<GATEWAY_IP>:/tmp/<GATEWAY_EUI>.crt
scp <GATEWAY_MQTT_KEY_FILE> root@<GATEWAY_IP>:/tmp/<GATEWAY_EUI>.key
```

On the gateway:

```
cp /tmp/ca.crt /etc/mosquitto/certs/ca.crt
cp /tmp/<GATEWAY_EUI>.crt /etc/mosquitto/certs/<GATEWAY_EUI>.crt
cp /tmp/<GATEWAY_EUI>.key /etc/mosquitto/certs/<GATEWAY_EUI>.key

chown mosquitto:mosquitto /etc/mosquitto/certs/ca.crt \
  /etc/mosquitto/certs/<GATEWAY_EUI>.crt \
  /etc/mosquitto/certs/<GATEWAY_EUI>.key
chmod 0644 /etc/mosquitto/certs/ca.crt /etc/mosquitto/certs/<GATEWAY_EUI>.crt
chmod 0600 /etc/mosquitto/certs/<GATEWAY_EUI>.key
rm -f /tmp/ca.crt /tmp/<GATEWAY_EUI>.crt /tmp/<GATEWAY_EUI>.key
ls -lah /etc/mosquitto/certs/
```

On a protected **server staging** directory, `root:root 0600` for the private key is acceptable. On the **gateway**, a root-owned `0600` key is unreadable by Mosquitto; the active key must be readable only by the Mosquitto service account.

Expected active gateway directory: exactly `ca.crt`, `<GATEWAY_EUI>.crt`, `<GATEWAY_EUI>.key`; none are zero bytes.

3. Verify certificate contents and key match

```
openssl x509 -in /etc/mosquitto/certs/ca.crt -noout -subject -issuer
openssl x509 -in /etc/mosquitto/certs/<GATEWAY_EUI>.crt -noout -subject -issuer -dates

CERT_HASH="$(openssl x509 -in /etc/mosquitto/certs/<GATEWAY_EUI>.crt -pubkey -noout | openssl pkey -pubin -
outform DER | sha256sum | cut -d' ' -f1)"
KEY_HASH="$(openssl pkey -in /etc/mosquitto/certs/<GATEWAY_EUI>.key -pubout -outform DER | sha256sum | cut -
d' ' -f1)"
echo "CERT: $CERT_HASH"
echo "KEY : $KEY_HASH"
```

The client certificate subject must be `CN = <GATEWAY_EUI>` and the hashes must match.

Step 7: Create the Mosquitto configuration file

What this step does

Writes `/etc/mosquitto/mosquitto.conf` for the loopback listener/persistence and `/etc/mosquitto/conf.d/bridge.conf` for the two mTLS bridges (cloud-uplink QoS 1 and cloud-downlink QoS 0).

Why we do it

Binds the local listener strictly to loopback (127.0.0.1) so local services can connect anonymously without exposing port 1883 to the network, and creates persistent outbound mTLS bridges for reliable cloud communication.

Procedure

Back up an existing static configuration:

```
[ ! -f /etc/mosquitto/mosquitto.conf ] || cp /etc/mosquitto/mosquitto.conf
/etc/mosquitto/mosquitto.conf.before-buffer
```

Create /etc/mosquitto/mosquitto.conf for the loopback listener and persistence:

```
mkdir -p /etc/mosquitto/conf.d
cat > /etc/mosquitto/mosquitto.conf <<'EOF'
user mosquitto
persistence true
persistence_location /etc/mosquitto/data/
persistence_file mosquitto.db
autosave_interval 60
autosave_on_changes false
max_queued_messages 100000
max_queued_bytes 104857600
max_inflight_messages 20
queue_qos0_messages false
log_dest syslog
connection_messages true
log_type error
log_type warning
log_type notice
listener 1883 127.0.0.1
protocol mqtt
allow_anonymous true
include_dir /etc/mosquitto/conf.d
EOF
```

Create /etc/mosquitto/conf.d/bridge.conf:

```
cat > /etc/mosquitto/conf.d/bridge.conf <<'EOF'
connection cloud-uplink
address <MQTT_BROKER_FQDN>:8883
bridge_protocol_version mqttv311
remote_clientid gw-up-<GATEWAY_EUI>
cleansession false
start_type automatic
restart_timeout 5 60
keepalive_interval 30
notifications false
try_private false
bridge_cafile /etc/mosquitto/certs/ca.crt
bridge_certfile /etc/mosquitto/certs/<GATEWAY_EUI>.crt
bridge_keyfile /etc/mosquitto/certs/<GATEWAY_EUI>.key
bridge_insecure false
topic as923/gateway/<GATEWAY_EUI>/event/# out 1
topic as923/gateway/<GATEWAY_EUI>/state/# out 1

connection cloud-downlink
address <MQTT_BROKER_FQDN>:8883
bridge_protocol_version mqttv311
remote_clientid gw-down-<GATEWAY_EUI>
cleansession true
start_type automatic
restart_timeout 5 60
keepalive_interval 30
notifications false
```

```
try_private false
bridge_cafile /etc/mosquitto/certs/ca.crt
bridge_certfile /etc/mosquitto/certs/<GATEWAY_EUI>.crt
bridge_keyfile /etc/mosquitto/certs/<GATEWAY_EUI>.key
bridge_insecure false
topic as923/gateway/<GATEWAY_EUI>/command/# in 0
EOF
```

The bridge hostname must appear in the broker certificate SAN. If local DNS does not resolve the approved SAN name, add a gateway /etc/hosts mapping. For the working lab certificate, which contains DNS:lora-test-server but not 192.168.8.50, the gateway uses:

```
printf '%s\n' '192.168.8.50 lora-test-server' >> /etc/hosts
```

and both bridge addresses use lora-test-server:8883. Do not use an IP absent from the SAN and do not set bridge_insecure true.

Verify both active files and replace all placeholders:

```
grep -n '<[^>]*>' /etc/mosquitto/mosquitto.conf /etc/mosquitto/conf.d/bridge.conf || true
```

Step 8: Test the configuration in the foreground

What this step does

Executes `mosquitto -c /etc/mosquitto/mosquitto.conf -v` in the terminal foreground to test parsing, listener binding, and mTLS certificate handshake.

Why we do it

Catches configuration syntax errors, certificate mismatch issues, or network routing errors in real time before starting the daemon in the background.

Procedure

Stop the service if it is running:

```
/etc/init.d/mosquitto stop
mosquitto -c /etc/mosquitto/mosquitto.conf -v
```

Because this configuration uses `log_dest syslog`, foreground output can appear blank after Mosquitto reports that it loaded `conf.d/bridge.conf`. That is not a hang. In another SSH session inspect:

```
logread -e mosquitto | tail -n 100
# or live:
logread -f -e mosquitto
```

Pass when the config parses, the listener binds to `127.0.0.1:1883`, both bridges complete mTLS, and normal keepalive `PINGREQ` / `PINGRESP` messages continue.

Step 8A: Isolate TCP, TLS, and MQTT authorization

These are three different tests. Run them in this order when the bridge cannot connect.

A. Raw TCP reachability

```
nc -w 3 <MQTT_BROKER_FQDN> 8883 </dev/null
echo $?
```

Exit 0 proves only route + TCP port reachability. Binary output from nc against a TLS port is normal.

B. TLS + hostname + client-certificate verification

```
openssl s_client \
  -connect <MQTT_BROKER_FQDN>:8883 \
  -servername <MQTT_BROKER_FQDN> \
  -verify_hostname <MQTT_BROKER_FQDN> \
  -verify_return_error \
  -CAfile /etc/mosquitto/certs/ca.crt \
  -cert /etc/mosquitto/certs/<GATEWAY_EUI>.crt \
  -key /etc/mosquitto/certs/<GATEWAY_EUI>.key \
  </dev/null
```

Pass evidence includes Verification: OK, Verified peername: <MQTT_BROKER_FQDN>, and Verify return code: 0 (ok).

C. MQTT authentication + ACL

```
mosquitto_pub \
  -h <MQTT_BROKER_FQDN> -p 8883 \
  --cafile /etc/mosquitto/certs/ca.crt \
  --cert /etc/mosquitto/certs/<GATEWAY_EUI>.crt \
  --key /etc/mosquitto/certs/<GATEWAY_EUI>.key \
  -i gw-cert-test-<GATEWAY_EUI> \
  -t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/state/test' \
  -m 'mtls-auth-test' -q 1 -d
```

CONNACK (0) proves MQTT authentication. PUBACK (... RC:0) proves the ACL allowed publication.

Step 9: Enable and start Mosquitto

What this step does

Enables Mosquitto at boot (/etc/init.d/mosquitto enable), restarts the service, and verifies process status and port binding (127.0.0.1:1883).

Why we do it

Ensures the persistent local buffer runs automatically on every boot and verifies that port 1883 is securely bound only to local loopback.

Procedure

Run over SSH:

```
/etc/init.d/mosquitto enable
/etc/init.d/mosquitto restart
sleep 3
ps w | grep '[m]osquitto'
```

Verify local loopback listener binding:

```
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

Confirm that the output shows 127.0.0.1:1883.

Check persistence directory creation:

```
ls -ld /etc/mosquitto/data /etc/mosquitto/certs
ls -l /etc/mosquitto/data
```

Step 10: Include broker files in Gateway OS backups

What this step does

Appends /etc/mosquitto/ to /etc/sysupgrade.conf.

Why we do it

Guarantees that Mosquitto configuration files, persistent queue settings, and TLS certificates are included in OpenWrt sysupgrade backups and survive future Gateway OS firmware updates.

Procedure

Run over SSH:

```
grep -qxF '/etc/mosquitto/' /etc/sysupgrade.conf 2>/dev/null || printf '%s\n' '/etc/mosquitto/' >>
/etc/sysupgrade.conf
cat /etc/sysupgrade.conf
```

Generate a fresh LuCI backup archive (**System > Backup / Flash Firmware > Generate archive**) and save it in a secure location off-device.

Step 11: Capture an off-device configuration integrity baseline

What this step does

Creates fingerprints for the gateway broker configuration and public certificate identity and copies the results off the gateway. This detects later configuration drift; it does **not** prove that queued message contents were never changed.

Why we do it

If an attacker or accidental change modifies the bridge endpoint, topic rules, listener binding, or certificate files, a baseline retained somewhere other than the Raspberry Pi gives the operator an independent value to compare against. A checksum kept only beside the file it protects is not an independent security reference.

Procedure

On the gateway, create a sanitized baseline file without copying the private key itself:

```
umask 077
{
  printf 'gateway_eui=%s\n' '<GATEWAY_EUI>'
  printf 'captured_utc=%s\n' "$(date -u +%Y-%m-%dT%H:%M:%SZ)"
  sha256sum /etc/mosquitto/mosquitto.conf /etc/mosquitto/conf.d/bridge.conf
  openssl x509 -in /etc/mosquitto/certs/<GATEWAY_EUI>.crt -noout -fingerprint -sha256 -serial -dates
  openssl x509 -in /etc/mosquitto/certs/ca.crt -noout -fingerprint -sha256 -subject -issuer
} > /tmp/gateway-mqtt-security-baseline.txt
cat /tmp/gateway-mqtt-security-baseline.txt
```

On the administration workstation, copy the baseline into the protected gateway records:

```
scp root@<GATEWAY_IP>:/tmp/gateway-mqtt-security-baseline.txt ./<GATEWAY_EUI>-mqtt-security-baseline.txt
```

Then remove the temporary gateway copy:

```
rm -f /tmp/gateway-mqtt-security-baseline.txt
```

Store the workstation copy with the approved Gateway OS image hash, configuration backup, certificate issuance record, and recovery documentation. During later verification, recalculate the same values and compare them with this off-device baseline. A mismatch requires investigation before normal service resumes.

Troubleshooting

Mosquitto starts with generated UCI file instead of static config

Inspect `/etc/config/mosquitto` and ensure option `use_uci '0'` is set.

Permission denied for private key or data folder

Ensure `/etc/mosquitto/certs/<GATEWAY_EUI>.key` is owned by `mosquitto:mosquitto` with mode `0600`. A server-staging `root:root 0600` key must not be copied onto the gateway without changing ownership.

Bridge reports connection refused or unknown CA

Verify system time (`date -u`), DNS resolution (`nslookup <MQTT_BROKER_FQDN>`), and certificate file paths.

Completion check

Before moving to the next manual, verify that: - [] Mosquitto runs from `/etc/mosquitto/mosquitto.conf`. - [] Port 1883 listens exclusively on `127.0.0.1`. - [] Both outbound mTLS bridges connect successfully over port 8883. - [] Persistent queue data directory `/etc/mosquitto/data/` exists on overlay storage. - [] `/etc/mosquitto/` is added to `/etc/sysupgrade.conf`. - [] An off-device configuration/certificate integrity baseline has been captured. - [] The operator understands that `mosquitto.db` is a bounded availability queue and is not claimed to be root-tamper-proof.

Next step

Continue with [04a-configure-gateway-integrity-journal.md](#) to add the independent evidence path, then configure MQTT Forwarder in [05-configure-mqtt-forwarder.md](#).

Supporting runbook: 05-configure-mqtt-forwarder.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Gateway 5. Configure ChirpStack MQTT Forwarder

MQTT Forwarder receives packets from Concentrator and publishes ChirpStack gateway messages to the local Mosquitto broker. It is the **delivery path**, not the gateway evidence authority.

The independently reviewed journal from Gateway 4A consumes the supported Concentrator event interface beside MQTT Forwarder. This separation lets the server later compare what the journal recorded with what MQTT actually delivered.

For this setup, MQTT Forwarder must not connect directly to the remote broker. The local broker provides TLS, topic bridging, and persistent uplink buffering.

Complete [04-configure-local-mqtt-buffer.md](#) and [04a-configure-gateway-integrity-journal.md](#) before final acceptance.

Before you begin

Have these values ready:

```
Gateway EUI: <GATEWAY_EUI>
Region topic prefix: <REGION_TOPIC_PREFIX>
Local broker address: tcp://127.0.0.1:1883
Forwarder client ID: gw-fwd-<GATEWAY_EUI>
```

Confirm the local broker is running:

```
ps w | grep '[m]osquitto'
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

Continue only when port 1883 is bound to 127.0.0.1.

Step 1: Open the MQTT Forwarder page

1. Sign in to the Gateway OS web interface.
2. Open **ChirpStack > MQTT Forwarder**.
3. Select the forwarder instance connected to the configured RAK5146.

The exact field names can vary between Gateway OS releases. Match the settings by purpose rather than copying an option name from another release.

Step 2: Enable the Concentratord backend

Set the backend fields to the equivalent of:

```
Enabled: yes
Backend: concentratord
```

Do not select `semtech_udp`. Concentratord is already controlling the RAK5146 and provides the packet stream used by MQTT Forwarder.

Step 3: Configure packet filtering

Use these normal forwarding settings:

```
Forward CRC OK packets: enabled
Forward CRC invalid packets: disabled
Forward packets without CRC: disabled unless required by the confirmed radio path
```

CRC-invalid packets are useful only for controlled RF diagnostics. Forwarding them during normal operation adds unusable traffic and can hide radio problems.

When the installed release provides a filter for non-LoRaWAN payloads, keep the setting consistent with the intended network. A normal LoRaWAN-only gateway can filter proprietary non-LoRaWAN frames.

Step 4: Configure the local MQTT connection

Set:

```
Server: tcp://127.0.0.1:1883
Topic prefix: <REGION_TOPIC_PREFIX>
QoS: 1
Clean session: false
Client ID: gw-fwd-<GATEWAY_EUI>
Use JSON: disabled
Username: empty
Password: empty
CA certificate: empty
Client certificate: empty
Private key: empty
```

Why these values are used

- 127.0.0.1 keeps the first MQTT hop inside the gateway.
- QoS 1 makes the local broker acknowledge gateway event and state messages.
- A fixed client ID prevents every restart from appearing as a different client.
- A non-clean session supports reliable reconnect behavior between MQTT Forwarder and the local broker.
- JSON is disabled because ChirpStack uses the Protobuf gateway message format on this path.
- TLS fields are empty because the loopback connection does not leave the gateway. The local Mosquitto bridge holds the remote TLS files.

Do not enter `<MQTT_BROKER_FQDN>` on this page. Doing so bypasses the persistent local queue and can also create unexplained disagreement with the independently designed journal/reconciliation path.

Step 5: Review metadata and command topics

When the page exposes event, command, state, or metadata topic templates, keep the standard ChirpStack gateway structure under the exact region prefix:

```
<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/...  
<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/state/...  
<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/command/...
```

Do not remove the Gateway EUI level or replace it with a hostname. The remote broker ACL and ChirpStack gateway lookup depend on this topic structure.

Step 6: Save the configuration

1. Recheck the Gateway EUI and region prefix.
2. Select **Save & Apply**.
3. Wait for MQTT Forwarder to restart.
4. Refresh the page and confirm the saved values remain present.

Step 7: Inspect the effective UCI configuration

Run over SSH:

```
uci show chirpstack-mqtt-forwarder  
monit status
```

Confirm that the effective configuration shows:

- the Concentrator backend;
- tcp://127.0.0.1:1883;
- <REGION_TOPIC_PREFIX>;
- QoS 1;
- the fixed gw-fwd-<GATEWAY_EUI> client ID;
- JSON disabled;
- no remote CA, certificate, or key path.

If the LuCI page does not expose QoS, clean-session, or client-ID fields, inspect the installed package before changing anything:

```
opkg list-installed | grep 'chirpstack-mqtt-forwarder'  
ps w | grep '[c]hirpstack-mqtt-forwarder'  
uci show chirpstack-mqtt-forwarder
```

Use only options supported by that package and its generated configuration. Do not guess UCI keys or edit the init script.

Step 8: Verify a local gateway event

Open one SSH session and subscribe to this gateway's event topics:

```
mosquitto_sub -h 127.0.0.1 -p 1883 \
-t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/#' -v
```

Generate one real uplink from an approved LoRaWAN test device.

The subscription should display a topic for the configured Gateway EUI. The payload is binary Protobuf and may appear unreadable in the terminal; the important check here is that a message arrives on the correct topic.

Press **Ctrl+C** after the test.

No message means the failure is before the remote bridge. Check Concentratord, the region prefix, the Gateway EUI, and the local broker before changing remote certificates.

Step 9: Verify the local listener remains private

Run:

```
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

The listener must remain 127.0.0.1:1883. The empty username and password fields are safe only for this loopback-only listener.

Step 10: Verify UDP Forwarder remains disabled

Run:

```
uci show chirpstack-udp-forwarder
```

Remove any enabled remote UDP server through **ChirpStack > UDP Forwarder**. Do not keep UDP as a second path or fallback.

Troubleshooting

MQTT Forwarder reports connection refused

Check Mosquitto first:

```
ps w | grep '[m]osquitto'
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

Start or repair the local broker. Do not replace 127.0.0.1 with the remote broker address.

No local event appears after an uplink

Check the path in order:

1. Concentratord has a stable Gateway EUI.
2. The device and gateway use the same region and channel plan.
3. MQTT Forwarder uses the Concentratord backend.
4. The topic prefix matches the active region.
5. The local Mosquitto listener is running.

Use the forwarder service messages only after checking the saved settings:

```
logread -e chirpstack-mqtt-forwarder
```

The topic uses the wrong region prefix

Return to Concentratord, confirm the exact channel plan, then update MQTT Forwarder and the local Mosquitto bridge to the same <REGION_TOPIC_PREFIX>. The remote ChirpStack server must enable that same region.

MQTT Forwarder repeatedly restarts

Compare the UCI values with the installed package's supported fields. Remove unsupported guessed options and reapply the configuration through LuCI.

Local events work but ChirpStack sees nothing

MQTT Forwarder and Concentratord are working. Continue troubleshooting at the local Mosquitto bridge, remote certificate, remote ACL, broker subscription, and ChirpStack region.

Completion check

Before continuing:

- MQTT Forwarder uses the Concentratord backend;
- the server is `tcp://127.0.0.1:1883`;
- QoS 1 is enabled;
- the client ID is fixed and gateway-specific;
- the topic prefix matches the radio and ChirpStack region;
- JSON is disabled;
- a real local event appears under the correct Gateway EUI;
- no remote TLS files are configured in MQTT Forwarder;
- UDP Forwarder is disabled;
- the evidence journal remains a separate read-only Concentratord consumer and MQTT Forwarder has no permission to rewrite its files.

Next step

Continue with [06-verify-gateway-os.md](#).

Supporting runbook: 08-mqtt-and-valkey.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

8. MQTT and Valkey HA on the Three-Node POC

Status: CORE SERVICE LAYER COMPLETE / VALIDATED. Mosquitto 2.0.18 is commissioned on `u1c-01` and `u1c-02` with TLS backend listeners on `:8884`, and the validated MQTT HAProxy TLS passthrough endpoint is `10.104.0.2:8883` using broker certificate identity `mqtt.internal.lorawan.com`; broker-backend failure and recovery have passed. Valkey 7.2.13 is commissioned TLS-only on all three nodes with authenticated replication, three TLS Sentinels (quorum 2), a least-privilege HAProxy health identity, dual writable-primary HAProxy endpoints `10.104.0.2:16379` and `10.104.0.4:16379`, and automatic promotion/rejoin testing. The most recent **Phase 8C.14 commissioning failover checkpoint** elected `u1c-02 (10.104.0.4)` as Valkey primary. Treat that as dated acceptance evidence, not a claim about today's elected primary; do not manually fail back solely to match documentation. ChirpStack workload MQTT authentication/ACLs and the second application-node MQTT routing boundary are intentionally deferred to Phase 9 before first ChirpStack start.

Execution-record authority: sections 8.1-8.17 preserve the original target design and are useful for rationale, but they contain placeholders and assumptions that were superseded during live commissioning. When those planning sections differ from the observed deployment, sections 8.18-8.21 and the live-state summary above are authoritative. In particular, the commissioned Valkey TLS identity is `valkey.internal.lorawan.com`, not `valkey-ha.internal.<DOMAIN>`, and the commissioned MQTT path is currently `10.104.0.2:8883 -> Mosquitto :8884`, not the uncommissioned `mqtt-ha.internal.<DOMAIN>:18883` design.

This manual deploys the two shared runtime dependencies used by ChirpStack:

```
MQTT transport      -> Mosquitto-1 preferred / Mosquitto-2 backup
fast shared state   -> Valkey primary + 2 replicas, managed by 3 Sentinels
```

The physical gateway's local Mosquitto queue remains separate on the Raspberry Pi. Do not confuse that edge buffer with the two cloud brokers.

8.1 Port plan and why the broker backend uses 8884

Use this exact separation:

```
PUBLIC MQTT SERVICE
mqtt.<DOMAIN>:8883
-> DigitalOcean Reserved IPv4
-> current owner: HAProxy on ha-01 OR ha-02 anchor :8883
-> Mosquitto backend TLS :8884

PRIVATE MQTT SERVICE
mqtt-ha.internal.<DOMAIN>:18883
-> local HAProxy on ha-01/ha-02/ha-03
-> same Mosquitto backend TLS :8884
```

8883 belongs to the HAProxy public frontend on the **anchor IP** of ha-01/02. The Mosquitto processes listen on private backend port 8884, so the broker and public HAProxy listener never contend for the same address/port.

Why: HAProxy and Mosquitto cannot both bind the same `<HOST_PRIVATE_IP>:8883`. Using 8884 internally removes that collision while preserving the normal public MQTT endpoint `mqtt.<DOMAIN>:8883`.

The cloud brokers do not need a plaintext 1883 listener. The only plaintext 1883 in this architecture is the Raspberry Pi gateway-local loopback broker at `127.0.0.1:1883`.

8.2 Preconditions

Before installing either service, record:

```
ha-01 private IP:
ha-02 private IP:
ha-03 private IP:
DOMAIN:
confirmed ChirpStack region/topic prefix:
Mosquitto image/version/digest:
Valkey image/version/digest:
MQTT CA path:
Valkey CA path:
Valkey shared application secret reference:
Sentinel administration secret reference:
```

Required certificates already issued by the PKI step:

```
Mosquitto-1 server certificate
  SAN: mqtt.<DOMAIN>
  SAN: mqtt-ha.internal.<DOMAIN>
  SAN: ha-01 private name/IP as required

Mosquitto-2 server certificate
  SAN: mqtt.<DOMAIN>
  SAN: mqtt-ha.internal.<DOMAIN>
  SAN: ha-02 private name/IP as required

Valkey-1/2/3 server certificates
  SAN: valkey-ha.internal.<DOMAIN>
  SAN: each node's own private name/IP
```

MQTT client identities required for the POC:

```
one unique certificate per physical gateway
  CN = its 16-hex Gateway EUI

ChirpStack workload certificate(s)
  not reused by gateways

Node-RED workload certificate
  read-only application-uplink permission
```

Stop here. Do not continue if certificate SANs, private IPs, region prefix, or image versions are still placeholders.

8.3 Prepare Mosquitto on ha-01 and ha-02

Run this section on **ha-01 and ha-02 only**.

Create directories:

```
sudo install -d -m 750 /etc/lorawan-cloud/mosquitto
sudo install -d -m 750 /etc/lorawan-pki/mqtt
sudo install -d -m 750 /srv/mosquitto/data
sudo install -m 640 /dev/null /etc/lorawan-cloud/mosquitto/acl
```

Install the node's broker certificate/key and the gateway/workload CA using the owner/group required by the pinned Mosquitto image or package. Never copy a CA private key to a broker.

Create `/etc/lorawan-cloud/mosquitto/mosquitto.conf` on each broker host. Replace `<THIS_HOST_PRIVATE_IP>` with that host's VPC address:

```
persistence true
persistence_location /mosquitto/data/
autosave_interval 60
log_dest stdout

listener 8884 <THIS_HOST_PRIVATE_IP>
protocol mqtt
allow_anonymous false

cafile /mosquitto/config/certs/ca.crt
certfile /mosquitto/config/certs/server.crt
keyfile /mosquitto/config/certs/server.key
require_certificate true
use_identity_as_username true
tls_version tlsv1.2

acl_file /mosquitto/config/acl
```

`require_certificate true` makes the TLS listener require a valid client certificate. `use_identity_as_username true` maps the client-certificate Common Name to the MQTT username used by the ACL.

Do not add a cloud listener on 1883.

8.4 Create the MQTT ACL

For each gateway, add only its own topics. Example shape:

```
user <GATEWAY_EUI>
topic write <REGION_PREFIX>/gateway/<GATEWAY_EUI>/event/#
topic write <REGION_PREFIX>/gateway/<GATEWAY_EUI>/state/#
topic read <REGION_PREFIX>/gateway/<GATEWAY_EUI>/command/#
```

For ChirpStack, give the reviewed workload identity the gateway-backend permissions it actually needs:

```
user <CHIRPSTACK_MQTT_IDENTITY>
topic read <REGION_PREFIX>/gateway/+/event/#
topic read <REGION_PREFIX>/gateway/+/state/#
topic write <REGION_PREFIX>/gateway/+/command/#
topic write application/+/device/+/event/#
topic read application/+/device/+/command/#
```

For Node-RED:

```
user <NODE_RED_MQTT_IDENTITY>
topic read application/+/device/+/event/up
```

If the pinned ChirpStack configuration uses separate gateway-backend and application-integration certificates, split the ChirpStack ACL accordingly rather than broadening one identity.

Copy the **same reviewed ACL policy** to Mosquitto-1 and Mosquitto-2. Keep certificate/private-key files node-specific.

8.5 Run each Mosquitto backend

A containerized host-network example is below. Use the exact paths/user documented by the pinned image:

```
services:
  mosquitto:
    image: <PINNED_MOSQUITTO_IMAGE>
    network_mode: host
    restart: unless-stopped
    volumes:
      - /etc/lorawan-cloud/mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - /etc/lorawan-cloud/mosquitto/acl:/mosquitto/config/acl:ro
      - /etc/lorawan-pki/mqtt:/mosquitto/config/certs:ro
      - /srv/mosquitto/data:/mosquitto/data
```

Validate before start using the validation method supported by the pinned image, then:

```
sudo docker compose -f /etc/lorawan-cloud/mosquitto/compose.yml config --quiet
sudo docker compose -f /etc/lorawan-cloud/mosquitto/compose.yml up -d
sudo docker compose -f /etc/lorawan-cloud/mosquitto/compose.yml ps
sudo docker compose -f /etc/lorawan-cloud/mosquitto/compose.yml logs --since=5m --tail=100
sudo ss -lntp | grep ':8884'
```

Pass only when the listener is on the intended private address and no cloud :1883 listener exists.

8.6 Prove Mosquitto TLS and ACL before HAProxy

From an approved VPC host, use a staging client certificate:

```
mosquitto_pub \
  -h <MOSQUITTO_PRIVATE_IP> -p 8884 \
  --cafile <MQTT_CA> \
  --cert <STAGING_CLIENT_CERT> \
  --key <STAGING_CLIENT_KEY> \
  -t '<AUTHORIZED_TEST_TOPIC>' \
  -m 'mqtt-ha-preflight' -q 1
```

Then attempt one topic that the identity is **not** allowed to publish/read. The authorized operation must work and the cross-identity operation must be denied.

A successful TLS handshake alone is not enough; ACL direction must also be proven.

8.7 Add the HAProxy MQTT frontends

Use the **same preferred/backup ordering on every HAProxy host** so clients do not split across two unrelated brokers.

On ha-01 and ha-02, add both frontends:

```
frontend mqtt_public
  mode tcp
  bind <THIS_HOST_ANCHOR_IP>:8883
  default_backend mosquitto_active_standby
```

```

frontend mqtt_internal
    mode tcp
    bind <THIS_HOST_PRIVATE_IP>:18883
    default_backend mosquitto_active_standby

backend mosquitto_active_standby
    mode tcp
    option tcp-check
    default-server inter 2s fall 3 rise 2
    server mosquitto-1 <HA01_PRIVATE_IP>:8884 check
    server mosquitto-2 <HA02_PRIVATE_IP>:8884 check backup

```

On ha-03, add **only** the private internal frontend plus the same backend:

```

frontend mqtt_internal
    mode tcp
    bind <HA03_PRIVATE_IP>:18883
    default_backend mosquitto_active_standby

backend mosquitto_active_standby
    mode tcp
    option tcp-check
    default-server inter 2s fall 3 rise 2
    server mosquitto-1 <HA01_PRIVATE_IP>:8884 check
    server mosquitto-2 <HA02_PRIVATE_IP>:8884 check backup

```

The runtime HAProxy check is intentionally a Layer-4 availability check. End-to-end certificate and ACL correctness is proven separately with actual MQTT clients because the broker requires client certificates.

Validate and reload on each affected host:

```

sudo haproxy -c -V -f /etc/haproxy/haproxy.cfg
sudo systemctl reload haproxy
sudo ss -lntp | grep -E ':(8883|18883)\b'

```

Expected:

```

ha-01: 8883 + 18883
ha-02: 8883 + 18883
ha-03: 18883 only

```

8.8 MQTT service naming: target design versus commissioned state

The original design below used a local-per-host service name on :18883. That route was **not** commissioned during Phase 8B and must not be treated as live:

```

mqtt-ha.internal.<DOMAIN>:18883    # target only; not commissioned

```

The currently validated cloud MQTT path is:

```

client TLS hostname/SNI: mqtt.internal.lorawan.com
HAProxy frontend:       10.104.0.2:8883
Mosquitto backends:     10.104.0.2:8884 and 10.104.0.4:8884
TLS termination:        Mosquitto, not HAProxy

```

HAProxy forwards MQTT TLS bytes unchanged, so clients must verify the certificate presented by the selected Mosquitto backend. Phase 9 must not start ChirpStack until it closes two remaining items: a ChirpStack-specific MQTT authentication/ACL identity, and a routing design that does not make the second ChirpStack instance depend on a single u1c-01 HAProxy process. If a local-per-app-node MQTT frontend is commissioned later, its

hostname must match an actually issued broker certificate identity; do not reuse the obsolete `mqtt-ha.internal.<DOMAIN>` placeholder unless new certificates explicitly contain it.

8.9 MQTT failover rehearsal

Before moving on to ChirpStack HA:

1. verify Mosquitto-1 is the selected backend from all three HAProxy hosts;
2. connect one staging MQTT client through `mqtt-ha.internal.<DOMAIN>:18883`;
3. publish/receive one authorized test message;
4. stop **Mosquitto-1 only**;
5. record the stop timestamp;
6. wait for HAProxy to mark it unavailable;
7. reconnect the staging client without changing hostname, certificate, or port;
8. prove Mosquitto-2 serves the connection;
9. restart Mosquitto-1;
10. verify it is healthy and returns to preferred status;
11. confirm no broker is unintentionally listening on cloud 1883.

Later, repeat the same failure using the physical gateway and its local QoS 1 queue.

Do not call this broker-state replication. Mosquitto-1 and Mosquitto-2 do not share sessions or queues.

8.10 Prepare Valkey storage and certificates

Run on **all three hosts**:

```
sudo install -d -m 750 /etc/lorawan-cloud/valkey
sudo install -d -m 750 /etc/lorawan-pki/valkey
sudo install -d -m 700 /srv/valkey/data
sudo install -d -m 750 /srv/valkey/sentinel
```

Use one protected long random application/replication secret for this tiny POC and a separate Sentinel-administration secret. Keep them outside Git and shell history.

8.11 Configure the initial Valkey primary and replicas

Use `ha-01` as the **bootstrap primary only**. Sentinel may later promote either replica.

Common secure baseline, adapted per host:

```
bind <THIS_HOST_PRIVATE_IP>
protected-mode yes
port 0
tls-port 6379

tls-cert-file /run/valkey-tls/server.crt
tls-key-file /run/valkey-tls/server.key
tls-ca-cert-file /run/valkey-tls/ca.crt
tls-auth-clients no
tls-replication yes

requirepass <VALKEY_SECRET>
masterauth <VALKEY_SECRET>
```

```
appendonly yes
appendfsync everysec
maxmemory 128mb
maxmemory-policy noeviction
```

On ha-02 and ha-03, additionally configure:

```
replicaof <HA01_PRIVATE_IP> 6379
```

Do not configure replicaof on the bootstrap primary ha-01.

noeviction is deliberate for the POC: memory exhaustion should fail visibly instead of silently discarding state.

If the pinned Valkey version uses ACLs instead of the password-only baseline, use a named least-privilege application/replication user and configure Sentinel authentication consistently. Do not mix incompatible ACL/password schemes accidentally.

8.12 Configure Sentinel on all three hosts

Create a writable sentinel.conf per host. Sentinel rewrites this file as topology changes, so do not mount it read-only.

Baseline:

```
bind <THIS_HOST_PRIVATE_IP>
protected-mode yes
port 0
tls-port 26379

tls-cert-file /run/valkey-tls/server.crt
tls-key-file /run/valkey-tls/server.key
tls-ca-cert-file /run/valkey-tls/ca.crt
tls-auth-clients no
tls-replication yes

requirepass <SENTINEL_SECRET>

sentinel monitor lorawan-valkey <HA01_PRIVATE_IP> 6379 2
sentinel auth-pass lorawan-valkey <VALKEY_SECRET>
sentinel down-after-milliseconds lorawan-valkey 5000
sentinel failover-timeout lorawan-valkey 60000
sentinel parallel-syncs lorawan-valkey 1
```

The 2 in sentinel monitor is the failure-detection quorum. Actual failover also needs a majority of the three Sentinel processes to authorize a leader for the failover.

Keep the Sentinel configuration/state path persistent and writable.

8.13 Run Valkey and Sentinel

Use the pinned Valkey image or package. If containerized, use stable host networking so Sentinel advertises the real VPC addresses instead of NAT-remapped container addresses.

A per-host container intent is:

```
services:
  valkey:
    image: <PINNED_VALKEY_IMAGE>
```

```

network_mode: host
restart: unless-stopped
command: ["valkey-server", "/etc/valkey/valkey.conf"]
volumes:
  - /etc/lorawan-cloud/valkey/valkey.conf:/etc/valkey/valkey.conf:ro
  - /etc/lorawan-pki/valkey:/run/valkey-tls:ro
  - /srv/valkey/data:/data

sentinel:
  image: <PINNED_VALKEY_IMAGE>
  network_mode: host
  restart: unless-stopped
  command: ["valkey-server", "/var/lib/valkey-sentinel/sentinel.conf", "--sentinel"]
  volumes:
    - /srv/valkey/sentinel:/var/lib/valkey-sentinel
    - /etc/lorawan-pki/valkey:/run/valkey-tls:ro

```

Place the host's writable `sentinel.conf` under `/srv/valkey/sentinel/` with ownership matching the image runtime user.

Validate the pinned image's binary/config paths before start.

8.14 Verify Valkey replication and Sentinel quorum

From an approved host with `valkey-cli`, use TLS and protected password input.

Check each data node:

```

valkey-cli --tls \
--cacert <VALKEY_CA> \
-h <VALKEY_PRIVATE_IP> -p 6379 \
-a '<LOAD_FROM_PROTECTED_SOURCE>' ROLE

```

Normal initial state:

```

ha-01 -> master/primary
ha-02 -> slave/replica
ha-03 -> slave/replica

```

Then query each Sentinel:

```

valkey-cli --tls \
--cacert <VALKEY_CA> \
-h <SENTINEL_PRIVATE_IP> -p 26379 \
-a '<LOAD_SENTINEL_SECRET_PROTECTED>' \
SENTINEL CKQUORUM lorawan-valkey

```

Pass only when all three Sentinels agree on the same primary and `CKQUORUM` reports enough Sentinels for quorum and failover authorization.

8.15 Validated HAProxy Valkey writable-primary routing

The live client-facing Valkey endpoints are:

```

ulc-01 -> 10.104.0.2:16379
ulc-02 -> 10.104.0.4:16379
TLS identity presented end-to-end -> valkey.internal.lorawan.com

```

Application traffic is TCP/TLS passthrough: HAProxy does **not** terminate or re-encrypt the client stream. HAProxy performs a separate TLS health connection to each Valkey backend, verifies the commissioned CA and SNI `valkey.internal.lorawan.com`, authenticates with the dedicated `haproxy-health` ACL identity loaded from a protected systemd EnvironmentFile, sends exact CRLF-framed `INFO replication`, and accepts only `role:master`.

The validated HAProxy 2.8 health-check shape is:

```
backend valkey_primary_backend
    mode tcp
    option tcp-check

    tcp-check connect ssl sni valkey.internal.lorawan.com
    tcp-check send-lf "AUTH haproxy-health %[env{VALKEY_HAPROXY_HEALTH_PASSWORD}]\r\n"
    tcp-check expect string +OK
    tcp-check send-lf "INFO replication\r\n"
    tcp-check expect min-recv 64 string role:master
```

The main Valkey application/replication password is **not** stored in `haproxy.cfg`. HAProxy receives only the separate least-privilege health credential. The `haproxy` service can read `/etc/haproxy/ca/valkey-ca.crt` but cannot read any Valkey node private key.

8.16 Validated Valkey endpoint and failover result

For a client on an application node, map `valkey.internal.lorawan.com` to that node's local HAProxy private IP and connect to `:16379`. A direct validation form is:

```
valkey-cli --tls \
--cacert <VALKEY_CA> \
--sni valkey.internal.lorawan.com \
-h <LOCAL_HAPROXY_PRIVATE_IP> -p 16379 \
-a '<LOAD_FROM_PROTECTED_SOURCE>' \
INFO replication
```

Expected routed role: `master`.

The controlled Phase 8C.14 failover proved the complete path: Sentinel promoted `ulc-02` after `ulc-03` Valkey was stopped; all three Sentinels agreed; both HAProxy `:16379` endpoints followed the new primary automatically without configuration changes or reloads; ten consecutive requests through each endpoint were master-only; pre-failover data survived; post-failover writes succeeded; `ulc-03` rejoined as a replica; and the final primary reported two connected replicas. At the Phase 8C.14 acceptance checkpoint, the Valkey primary was `ulc-02 (10.104.0.4)`. Treat Sentinel as authoritative for the live elected primary and do not manually fail back merely to match this dated checkpoint.

8.17 Final acceptance and Phase 9 handoff

Phase 8 core-service acceptance is split deliberately:

MQTT infrastructure accepted: Mosquitto 2.0.18 runs on `ulc-01/02`, both brokers serve TLS on `:8884`, `10.104.0.2:8883` provides validated HAProxy TLS passthrough, the shared broker certificate name `mqtt.internal.lorawan.com` verifies, HAProxy detects a failed broker, traffic can continue through the surviving broker, and service recovery has passed. This proves the broker/TLS/failover foundation only. ChirpStack workload authentication/ACLs, gateway identities, and a two-app-node MQTT route remain explicit later dependencies.

Valkey HA accepted: one primary and two replicas are maintained; three TLS Sentinels have quorum 2; failover-safe `masterauth` exists on every node; HAProxy exposes only the writable primary through both application-node

:16379 endpoints; automatic promotion, client routing convergence, data survival, writes after promotion, and old-primary replica recovery all passed.

Phase 9 may now begin with **read-only ChirpStack preflight and dependency closure**. Do not start ChirpStack until its exact image/config schema is pinned and the remaining MQTT workload identity/ACL plus second application-node routing boundary is closed.

8.18 Phase 8 Execution Record - MQTT Foundation Deployment

Recorded deployment evidence:

```
Date:
2026-08-24

MQTT baseline discovery:

ulc-01
10.104.0.2

ulc-02
10.104.0.4

ulc-03
10.104.0.8

Before installation:
- no MQTT packages installed
- no Valkey packages installed
- no MQTT listeners
- no Valkey listeners
- no MQTT/Valkey containers
- no MQTT/Valkey services

Mosquitto deployment:

Installed:
- mosquitto
- mosquitto-clients

Verified:
- Mosquitto version: 2.0.18
- MQTT protocol support: MQTT v5.0/v3.1.1/v3.1

Service state:

ulc-01:
mosquitto.service active (running)

ulc-02:
mosquitto.service active (running)

Result:
MQTT broker foundation installed successfully on the first two application nodes.
```

Next validation steps:

1. Configure Mosquitto TLS listeners.
2. Deploy MQTT CA and broker certificates.
3. Configure MQTT authentication and ACLs.
4. Validate broker-to-client TLS.
5. Add HAProxy MQTT routing.
6. Continue with Valkey deployment.

8.19 Phase 8 MQTT TLS Foundation Precheck Record

Executed on:

```
ulc-01  
10.104.0.2
```

```
ulc-02  
10.104.0.4
```

Discovery results:

```
Mosquitto:  
- installed  
- active  
- enabled  
  
MQTT PKI:  
- /etc/lorawan-pki/mqtt does not exist yet  
  
Existing PKI:  
- PostgreSQL TLS present  
- PgBouncer TLS present  
- MQTT TLS material not issued yet
```

Current listeners before TLS configuration:

```
ulc-01:  
127.0.0.1:1883  
[::1]:1883  
  
ulc-02:  
127.0.0.1:1883  
[::1]:1883
```

Interpretation:

- Mosquitto is running in the default local-only state.
- No cloud MQTT listener is exposed.
- No TLS listener exists yet.
- No plaintext remote MQTT exposure exists.

Next execution step:

1. Issue MQTT broker certificates from the project CA.
2. Install /etc/lorawan-pki/mqtt on broker nodes.
3. Replace local-only Mosquitto configuration with TLS listener configuration.
4. Validate mutual TLS before HAProxy integration.

Operational note:

Gateway client certificate provisioning is intentionally deferred until the final onboarding stage. Cloud MQTT broker TLS identities must be completed, HAProxy routing must be validated, Valkey and ChirpStack integration must be operational, and production cloud-side testing must pass before issuing gateway certificates.

Certificate separation:

- Broker certificates remain only on cloud MQTT nodes.
- Gateway certificates are separate client identities.

- Broker private keys are never transferred to gateways.

Automation preference:

Deployment steps should use chained command blocks where safe. Validation, backup, permission checks, and installation actions should be grouped into reproducible runbooks instead of many isolated commands.

Deployment ordering decision - Gateway certificate provisioning

Gateway MQTT client certificate provisioning is intentionally moved to the final deployment stage.

Reason:

- Cloud MQTT infrastructure must be completed and validated first.
- Mosquitto TLS, HAProxy routing, ACL policies, ChirpStack integration, and monitoring must exist before issuing gateway identities.
- Gateway certificates are production client credentials and should only be generated when the receiving MQTT service is ready.
- This prevents issuing gateway credentials against an unfinished broker architecture.

Final order:

1. Cloud foundation
 - etcd
 - Spilo/Patroni PostgreSQL HA
 - HAProxy
 - PgBouncer
2. Cloud application services
 - Mosquitto TLS
 - Valkey
 - ChirpStack
 - integrations
3. Cloud validation
 - TLS verification
 - failover testing
 - ACL testing
 - backup/recovery testing
4. Final gateway onboarding
 - issue gateway client certificate
 - install gateway CA/client certificate/key
 - configure MQTT Forwarder
 - validate end-to-end uplink

Gateway certificate provisioning must not begin before the cloud MQTT stack passes acceptance testing.

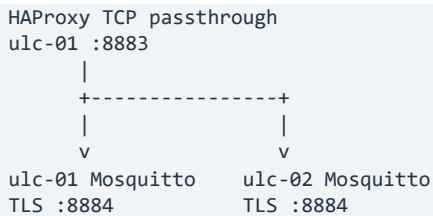
8.20 Phase 8B - MQTT HAProxy TLS Access Layer Execution Record

Objective:

Create a single MQTT TLS endpoint for gateways while keeping TLS termination on Mosquitto brokers.

Final design:

```
Gateway clients
|
| MQTT TLS :8883
|
```



Important design decision:

- HAProxy does not terminate MQTT TLS.
- Mosquitto retains broker certificates and private keys.
- HAProxy only forwards encrypted TCP traffic.
- Gateway certificates remain deferred until final onboarding.

8.20.1 HAProxy baseline check

Executed on ulc-01:

```
systemctl is-active haproxy
systemctl is-enabled haproxy
ss -H -lntp | grep haproxy
```

Existing services preserved:

```
PostgreSQL primary:
15432

PostgreSQL replicas:
15433
```

8.20.2 Initial HAProxy MQTT configuration attempt

The first configuration attempted:

```
frontend mqtt_tls
  bind 10.104.0.2:8883

backend mqtt_brokers
  server mqtt-ulc01 10.104.0.2:8883 check
  server mqtt-ulc02 10.104.0.4:8883 check
```

Validation passed:

```
haproxy -c -f /etc/haproxy/haproxy.cfg
```

Runtime failed because Mosquitto already owned port 8883:

```
cannot bind socket
Address already in use
```

Correction:

- HAProxy owns the external MQTT port.
- Mosquitto moves to an internal TLS port.

8.20.3 Move Mosquitto internal TLS listener

Mosquitto configuration changed:

Before:

```
listener 8883
```

After:

```
listener 8884

cafile /etc/lorawan-pki/mqtt/ca.crt
certfile /etc/lorawan-pki/mqtt/server.crt
keyfile /etc/lorawan-pki/mqtt/server.key

tls_version tlsv1.3
require_certificate false
```

Restart:

```
systemctl restart mosquitto
```

Validation:

```
ss -H -lntp | grep mosquitto
```

Expected:

```
0.0.0.0:8884
[::]:8884
```

8.20.4 Final HAProxy MQTT configuration

Backup first:

```
cp /etc/haproxy/haproxy.cfg \
/etc/haproxy/haproxy.cfg.phase8b-before-$(date -u +%Y%m%d-%H%M%S)
```

Configuration added:

```
frontend mqtt_tls
  bind 10.104.0.2:8883
  mode tcp
  option tcplog
  default_backend mqtt_brokers

backend mqtt_brokers
  mode tcp
  option tcp-check

  # The two Mosquitto brokers do not replicate live MQTT sessions/topic state.
  # Keep all normal sessions on ulc-01 so a gateway's separate uplink/downlink
  # connections and ChirpStack MQTT sessions cannot be split across brokers.
  server mqtt-ulc01 10.104.0.2:8884 check
  server mqtt-ulc02 10.104.0.4:8884 check backup
```

Validate:

```
haproxy -c -f /etc/haproxy/haproxy.cfg
```

Restart:

```
systemctl restart haproxy
```

Final listeners:

```
HAProxy:
10.104.0.2:8883

Mosquitto:
ulc-01 :8884
ulc-02 :8884
```

8.20.5 MQTT HA TLS validation

Client validation performed from ulc-03:

```
openssl s_client \
-connect 10.104.0.2:8883 \
-CAfile /etc/lorawan-pki/mqtt/ca.crt \
-verify_hostname mqtt.internal.lorawan.com \
-brief \
</dev/null
```

Result:

```
Protocol version: TLSv1.3
Cipher: TLS_AES_256_GCM_SHA384
Verification: OK
Verified peername: mqtt.internal.lorawan.com
```

PASS:

- HAProxy TLS passthrough
- broker certificate validation
- MQTT TLS endpoint

8.20.6 MQTT failover validation

Failure test:

Stop ulc-01 broker:

```
systemctl stop mosquitto
```

HAProxy detected failure:

```
Server mqtt_brokers/mqtt-ulc01 is DOWN
reason: Layer4 connection problem
```

Client test through HAProxy remained successful:

```
Verification: OK
```

Verified peername: mqtt.internal.lorawan.com

Recovery:

```
systemctl start mosquitto
```

HAProxy recovered the backend:

```
Server mqtt_brokers/mqtt-ulc01 is UP
reason: Layer4 check passed
2 active and 0 backup servers online
```

Phase 8B result:

MQTT HAProxy endpoint	PASS
TLS passthrough	PASS
ulc-01 Mosquitto backend	PASS
ulc-02 Mosquitto backend	PASS
Failover detection	PASS
Service recovery	PASS

8.21 Phase 8C - Valkey HA Deployment Execution Record

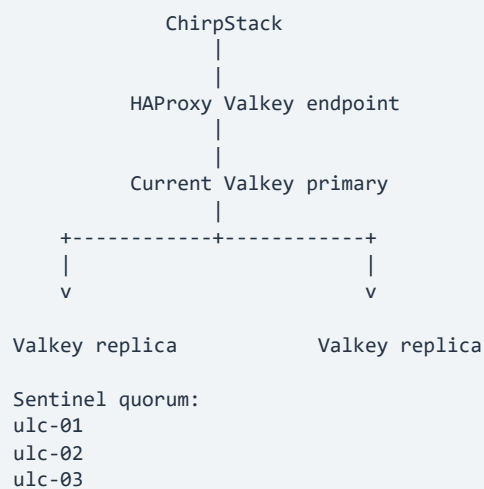
Objective:

Deploy the ChirpStack shared state backend using Valkey with future Sentinel-based high availability.

Execution boundary:

- Valkey deployment started after MQTT HA completion.
- Gateway provisioning remains deferred until cloud services are complete.
- No ChirpStack integration changes are performed until Valkey HA passes validation.
- No Git commit or push is performed during deployment sessions. Operator performs final review and commit.

Final intended architecture:



8.21.1 Valkey foundation precheck

Validated nodes:

```
ulc-01  
10.104.0.2  
  
ulc-02  
10.104.0.4  
  
ulc-03  
10.104.0.8
```

Validation performed:

```
Redis/Valkey packages:  
none installed  
  
Redis/Valkey services:  
none configured  
  
Listeners:  
6379 unavailable  
26379 unavailable
```

Resource validation:

```
ulc-01:  
RAM 1.9Gi  
Available 1.3Gi  
  
ulc-02:  
RAM 1.9Gi  
Available 1.4Gi  
  
ulc-03:  
RAM 1.9Gi  
Available 1.4Gi
```

Result:

```
Phase 8C.1 Foundation Precheck: PASS
```

8.21.2 Valkey package installation

Deployment automation:

```
ssh \  
-i /root/.ssh/cloud-deployment-phase8 \  
-o IdentitiesOnly=yes \  
opsadmin@NODE
```

The deployment key is required for multi-node automation. Do not rely on default SSH keys.

Installed package:

```
valkey-server  
valkey-tools
```

Version installed:

Valkey 7.2.13

8.21.3 Installation status checkpoint

ulc-01

Status:

PASS

Verified:

```
valkey-server installed
service active
service enabled
```

Listener:

```
127.0.0.1:6379
[::1]:6379
```

ulc-02

Status:

PASS

Verified:

```
valkey-server installed
service active
service enabled
```

Listener:

```
127.0.0.1:6379
[::1]:6379
```

ulc-03

Status:

PASS

The first remote installation attempt failed because SSHing from ulc-03 back into opsadmin@10.104.0.8 did not provide a TTY for sudo. The corrected procedure installed Valkey locally on ulc-03 and then performed a local verification.

Verified final state:

```
Valkey version: 7.2.13
service: active
startup: enabled
```

```
listener: 127.0.0.1:6379 and [::1]:6379
PING: PONG
```

Final Phase 8C.2 foundation state:

ulc-01	Valkey 7.2.13	active	enabled	loopback:6379	PONG
ulc-02	Valkey 7.2.13	active	enabled	loopback:6379	PONG
ulc-03	Valkey 7.2.13	active	enabled	loopback:6379	PONG

Result:

```
Phase 8C.2 Valkey Foundation: PASS
```

No node exposes Valkey on the private VPC yet. Replication, authentication, TLS/private binding, Sentinel, and HAProxy routing remain intentionally deferred to Phase 8C.3 and later.

8.21.4 Next execution step

Resume from ulc-03. Do not SSH from ulc-03 back into itself for privileged package installation. The failed first attempt occurred because remote sudo on opsadmin@10.104.0.8 required a TTY. Install the remaining package locally with normal interactive sudo, then use the existing Phase 8 deployment key only to verify ulc-01 and ulc-02.

Use this single automation block:

```
sudo -v

sudo bash -euo pipefail <<'EOF'

echo '=== PHASE 8C.2 COMPLETE VALKEY FOUNDATION ==='

SSH_KEY='/root/.ssh/cloud-deployment-phase8'

echo
echo '=== STEP 1 - VERIFY THIS IS ULC-03 ==='

[ "$(hostname)" = 'ulc-03' ] || {
    echo 'FAIL: run this block on ulc-03 only'
    exit 1
}

echo 'host = ulc-03: PASS'

echo
echo '=== STEP 2 - INSTALL VALKEY LOCALLY ON ULC-03 ==='

export DEBIAN_FRONTEND=noninteractive
apt-get update
apt-get install -y valkey-server valkey-tools

systemctl enable valkey-server
systemctl start valkey-server

echo
echo '=== STEP 3 - VERIFY ULC-03 ==='

valkey-server --version
systemctl is-active valkey-server
systemctl is-enabled valkey-server
ss -H -lnt | grep -E '127\.0\.0\.1:6379|\[::1\]:6379' || {
    echo 'FAIL: ulc-03 local Valkey listener missing'
    exit 1
}

valkey-cli -h 127.0.0.1 -p 6379 PING | grep -Fxq PONG || {
    echo 'FAIL: ulc-03 Valkey PING failed'
    exit 1
}
```

```

}

echo 'ulc-03 Valkey foundation: PASS'

echo
echo '=== STEP 4 - VERIFY ALL THREE NODES ==='

for ENTRY in \
    'ulc-01|10.104.0.2' \
    'ulc-02|10.104.0.4'
do
    NAME="${ENTRY%|*}"
    IP="${ENTRY##*|}"

    echo
    echo "===== $NAME ($IP) ====="

    ssh \
        -i "$SSH_KEY" \
        -o IdentitiesOnly=yes \
        -o StrictHostKeyChecking=yes \
        "opsadmin@$IP" '
            set -e
            hostname
            valkey-server --version
            systemctl is-active valkey-server
            systemctl is-enabled valkey-server
            ss -H -lnt | grep -E "127\.0\.0\.1:6379|\[::1\]:6379"
            valkey-cli -h 127.0.0.1 -p 6379 PING
        '
done

echo
echo '===== ulc-03 (10.104.0.8) ====='
hostname
valkey-server --version
systemctl is-active valkey-server
systemctl is-enabled valkey-server
ss -H -lnt | grep -E "127\.0\.0\.1:6379|\[::1\]:6379"
valkey-cli -h 127.0.0.1 -p 6379 PING

echo
echo '=== PHASE 8C.2 VALKEY FOUNDATION: PASS ==='
echo 'All three nodes have Valkey installed, active, enabled, local-only, and responding to PING.'
echo 'STOP HERE. Do not expose 6379 or configure replication until Phase 8C.3 security configuration.'

EOF

```

The block performs four clear actions:

1. Confirms it is running on ulc-03.
2. Installs and starts Valkey locally on ulc-03, avoiding the self-SSH sudo problem.
3. Verifies version, service state, loopback-only listener, and PONG on ulc-03.
4. Verifies the same accepted foundation state on ulc-01 and ulc-02 through the retained deployment SSH key.

Expected final foundation state:

ulc-01	Valkey 7.2.13	active	enabled	loopback:6379	PONG
ulc-02	Valkey 7.2.13	active	enabled	loopback:6379	PONG
ulc-03	Valkey 7.2.13	active	enabled	loopback:6379	PONG

Latest verified execution evidence:

```

ulc-01 (10.104.0.2)
Server v=7.2.13
active
enabled

```

```

127.0.0.1:6379 LISTEN
[::1]:6379 LISTEN
PING -> PONG
PASS

ulc-02 (10.104.0.4)
Server v=7.2.13
active
enabled
127.0.0.1:6379 LISTEN
[::1]:6379 LISTEN
PING -> PONG
PASS

```

At this checkpoint, ulc-01 and ulc-02 are definitively accepted. Do not mark Phase 8C.2 complete until the Step 3/Step 5 output for ulc-03 also shows Valkey 7.2.13, active, enabled, loopback-only 6379, and PONG.

After all three pass, continue to Phase 8C.3 in this order:

1. Run the security/TLS preflight below on all three nodes.
2. Issue and deploy Valkey server certificates only after confirming the actual Ubuntu package paths, service account, and TLS capability.
3. Configure authentication plus private-network/TLS listeners.
4. Configure one primary and two replicas.
5. Deploy three Sentinel members with quorum 2.
6. Add HAProxy writable-primary routing.
7. Perform controlled Valkey failover and recovery testing.

8.21.5 Phase 8C.3 security/TLS preflight

Observed preflight checkpoint:

```

ulc-01: Valkey 7.2.13, valkey:valkey, /etc/valkey/valkey.conf, loopback 6379, OpenSSL linked, PONG
ulc-02: Valkey 7.2.13, valkey:valkey, /etc/valkey/valkey.conf, loopback 6379, OpenSSL linked, PONG
ulc-03: Valkey 7.2.13, valkey:valkey, /etc/valkey/valkey.conf; output was interrupted by the systemctl pager
at "lines 1-4/4 (END)"

```

The ulc-03 interruption is not a Valkey failure. `systemctl show` opened the pager because this portion was executed locally in an interactive terminal. Press `q` to leave the pager, then run the no-pager continuation below. Do not mutate authentication, TLS listeners, replication, or Sentinel until the ulc-03 remainder is captured.

```

sudo bash -euo pipefail <<'EOF'

echo '=== PHASE 8C.3 ULC-03 PREFLIGHT CONTINUATION ==='

[ "$(hostname)" = 'ulc-03' ] || {
    echo 'FAIL: run on ulc-03 only'
    exit 1
}

echo
echo '--- systemd service identity ---'
SYSTEMD_PAGER=cat systemctl show valkey-server --no-pager \
    -p User \
    -p Group \
    -p FragmentPath \
    -p ExecStart

echo
echo '--- package configuration files ---'
dpkg -L valkey-server |
    grep -E '/etc/.*(valkey|redis).*\conf$|/etc/default/' || true

```

```

echo
echo '--- configured security/persistence directives ---'
grep -En \
'^[[:space:]]*(bind|protected-mode|port|tls-port|tls-cert-file|tls-key-file|tls-ca-cert-file|tls-auth-
clients|requirepass|masterauth|appendonly|dir|dbfilename)[[:space:]]' \
/etc/valkey/valkey.conf || true

echo
echo '--- relevant defaults including commented directives ---'
grep -En \
'^[[:space:]]*#[[:space:]]*(bind|protected-mode|port|tls-port|tls-cert-file|tls-key-file|tls-ca-cert-
file|tls-auth-clients|requirepass|masterauth|appendonly|dir|dbfilename)[[:space:]]' \
/etc/valkey/valkey.conf | head -80 || true

echo
echo '--- current listener ---'
ss -H -lntp | grep ':6379' || true

echo
echo '--- TLS capability evidence ---'
ldd "$(command -v valkey-server)" 2>/dev/null |
grep -Ei 'ssl|crypto' || true

echo
echo '--- local PING ---'
valkey-cli -h 127.0.0.1 -p 6379 PING

echo
echo '=== PHASE 8C.3 ULC-03 PREFLIGHT CONTINUATION COMPLETE ==='

EOF

```

Acceptance for this continuation is: service identity valkey:valkey, /etc/valkey/valkey.conf confirmed, loopback-only 6379, OpenSSL libraries linked, and PONG.

This step is read-only. It discovers the actual Valkey service account, configuration files, current bind/persistence settings, and TLS support before any security or replication mutation.

```

sudo -v
sudo bash -euo pipefail <<'EOF'
SSH_KEY='/root/.ssh/cloud-deployment-phase8'

echo '=== PHASE 8C.3 VALKEY SECURITY/TLS PREFLIGHT ==='

for ENTRY in 'ulc-01|10.104.0.2' 'ulc-02|10.104.0.4'; do
    NAME="${ENTRY%|*}"
    IP="${ENTRY##*|}"
    ssh -i "$SSH_KEY" -o IdentitiesOnly=yes -o StrictHostKeyChecking=yes "opsadmin@$IP" "bash -s -- '$NAME'"
<<'REMOTE'
set -euo pipefail
NAME="$1"
echo
echo "===== $NAME ====="
hostname
echo '--- package/version ---'
dpkg-query -W -f='${Status}|${Version}\n' valkey-server valkey-tools
valkey-server --version
echo '--- systemd ---'
systemctl show valkey-server -p User -p Group -p FragmentPath -p ExecStart
echo '--- config files ---'
dpkg -L valkey-server | grep -E '/etc/.*(valkey|redis).*\conf$|/etc/default/' || true
echo '--- active directives ---'
grep -Eh '^[[[:space:]]*(bind|protected-mode|port|tls-port|tls-cert-file|tls-key-file|tls-ca-cert-
file|requirepass|masterauth|appendonly|dir|dbfilename)[[:space:]]' /etc/valkey/*.conf /etc/valkey/*.conf.d/*
2>/dev/null || true
echo '--- listener ---'
ss -H -lntp | grep ':6379' || true
echo '--- TLS evidence ---'
ldd "$(command -v valkey-server)" 2>/dev/null | grep -Ei 'ssl|crypto' || true
valkey-server --help 2>&1 | grep -i tls || true
echo '--- ping ---'

```

```

valkey-cli -h 127.0.0.1 -p 6379 PING
REMOTE
done

echo
echo '==== ulc-03 ==== '
hostname
echo '--- package/version ---'
dpkg-query -W -f='${Status}]${Version}\n' valkey-server valkey-tools
valkey-server --version
echo '--- systemd ---'
systemctl show valkey-server -p User -p Group -p FragmentPath -p ExecStart
echo '--- config files ---'
dpkg -L valkey-server | grep -E '/etc/.*(valkey|redis).*\conf$|/etc/default/' || true
echo '--- active directives ---'
grep -Eh '^[:space:]]*(bind|protected-mode|port|tls-port|tls-cert-file|tls-key-file|tls-ca-cert-
file|requirepass|masterauth|appendonly|dir|dbfilename)[[:space:]]' /etc/valkey/*.conf /etc/valkey/*.conf.d/*
2>/dev/null || true
echo '--- listener ---'
ss -H -lntp | grep ':6379' || true
echo '--- TLS evidence ---'
ldd "$(command -v valkey-server)" 2>/dev/null | grep -Ei 'ssl|crypto' || true
valkey-server --help 2>&1 | grep -i tls || true
echo '--- ping ---'
valkey-cli -h 127.0.0.1 -p 6379 PING

echo
echo '=== PHASE 8C.3 PREFLIGHT COMPLETE ==='
EOF

```

Do not expose port 6379, set passwords, configure replication, or deploy Sentinel until this preflight output has been reviewed.

8.21.5 Phase 8C.3 security/TLS preflight result

The preflight passed on all three nodes.

Validated baseline:

```

Valkey server/tools: 7.2.13+dfsg1-0ubuntu0.1
Service user/group: valkey:valkey
Main config: /etc/valkey/valkey.conf
Systemd unit: /usr/lib/systemd/system/valkey-server.service
Current listener: loopback-only 6379
Local connectivity: PONG
TLS libraries: libssl.so.3 + libcrypto.so.3

```

The explicitly visible active baseline on ulc-03 is:

```

bind 127.0.0.1 -::1
protected-mode yes
port 6379
dbfilename dump.rdb
dir /var/lib/valkey
appendonly no

```

Result:

```

Phase 8C.3 security/TLS preflight: PASS

```

8.21.6 Phase 8C.4 Valkey TLS certificate issuance result

Certificate issuance completed successfully on ulc-03 from the existing internal CA.

Issuance directory:

```
/root/lorawan-pg-ca/valkey-issuance-20260825-003932
```

CA identity:

```
subject=CN = LoRaWAN PostgreSQL Internal CA
issuer=CN = LoRaWAN PostgreSQL Internal CA
SHA-256
fingerprint=99:00:4B:B3:2D:7D:78:FA:38:61:7C:78:89:6D:7A:7E:FF:9F:A6:10:FC:8F:07:D4:E2:5E:35:25:36:E6:CB:3E
```

Issued node identities:

```
ulc-01 -> valkey.internal.lorawan.com + ulc-01 + 10.104.0.2
ulc-02 -> valkey.internal.lorawan.com + ulc-02 + 10.104.0.4
ulc-03 -> valkey.internal.lorawan.com + ulc-03 + 10.104.0.8
```

Each certificate includes both `serverAuth` and `clientAuth` extended key usage so the same per-node identity can participate in TLS-protected Valkey replication and later Sentinel communication without sharing one private key across hosts.

Certificate-chain verification passed for all three nodes. Certificate/private-key public-key hashes also matched for all three nodes.

Recorded SHA-256 certificate fingerprints:

```
ulc-01 27:17:81:BA:E3:9A:8D:F8:AF:C0:CD:DA:1A:2D:DB:25:27:84:C4:65:B3:4D:B9:40:7A:32:89:F7:EB:AF:AC:0A
ulc-02 BD:DD:EF:C9:95:C7:CF:36:50:77:CE:42:38:97:B0:DB:77:CB:F3:2C:50:AD:41:98:9C:AA:D6:84:0A:43:17:A6
ulc-03 E5:5C:A8:BE:E1:C7:0D:9E:A8:5C:93:3B:4B:80:EF:B8:58:FD:47:FA:08:8C:5E:3C:6D:94:C4:4C:9F:B0:4B:DC
```

Result:

```
Phase 8C.4 Valkey TLS certificate issuance: PASS
```

Keep the complete issuance directory until the Valkey/Sentinel HA layer and final security cleanup are complete. Do not delete the deployment SSH key or node private keys during commissioning.

8.21.7 Phase 8C.5 Valkey TLS material installation checkpoint

The remote TLS material installation completed successfully on `ulc-01` and `ulc-02`.

Validated on both nodes:

```
/etc/lorawan-pki/valkey          750 root:valkey
/etc/lorawan-pki/valkey/ca.crt    640 root:valkey
/etc/lorawan-pki/valkey/server.crt 640 root:valkey
/etc/lorawan-pki/valkey/server.key 640 root:valkey

ca-read-PASS
cert-read-PASS
key-read-PASS
server.crt: OK
```

Observed file sizes on `ulc-01` and `ulc-02`:

```
ca.crt      1891 bytes
server.crt  1627 bytes
```

```
server.key 1704 bytes
```

Final Phase 8C.5 status:

```
ulc-01 TLS material: PASS
ulc-02 TLS material: PASS
ulc-03 TLS material: PASS
```

ulc-03 local verification confirmed:

```
/etc/lorawan-pki/valkey          750 root:valkey
/etc/lorawan-pki/valkey/ca.crt    640 root:valkey (1891 bytes)
/etc/lorawan-pki/valkey/server.crt 640 root:valkey (1627 bytes)
/etc/lorawan-pki/valkey/server.key 640 root:valkey (1708 bytes)
ca-read-PASS
cert-read-PASS
key-read-PASS
/etc/lorawan-pki/valkey/server.crt: OK
certificate/private-key hash match: PASS
```

The Valkey service was deliberately left unchanged after PKI staging. ulc-03 remained active, listening only on loopback plaintext 6379, and returned PONG. This proves certificate installation itself did not alter runtime behavior.

Result:

```
Phase 8C.5 Valkey TLS material installation: PASS
```

8.21.8 Phase 8C.6 Valkey TLS + authentication result

Phase 8C.6 passed on all three nodes.

Validated runtime state:

```
ulc-01 10.104.0.2:6379 TLS-only + password auth: PASS
ulc-02 10.104.0.4:6379 TLS-only + password auth: PASS
ulc-03 10.104.0.8:6379 TLS-only + password auth: PASS
```

Security checks passed on every node:

```
service active + enabled
TLSv1.3 negotiated
cipher TLS_AES_256_GCM_SHA384
CA verification OK
peer name valkey.internal.lorawan.com verified
plaintext connection rejected
TLS without password rejected with NOAUTH
TLS with password returned PONG
tls-replication yes prepared
```

The shared Valkey authentication secret is retained only in the root-owned secret file created during commissioning and was not printed in the execution record.

Result:

```
Phase 8C.6 Valkey TLS + authentication: PASS
```

8.21.9 Phase 8C.7 initial replication attempt and authentication correction

The first Phase 8C.7 replication block stopped before any replica configuration was changed. Its primary precheck attempted to authenticate `valkey-cli` using the `VALKEYCLI_AUTH` environment variable, but the observed Ubuntu package behavior returned:

```
exit_code=0
NOAUTH Authentication required.
```

This is a command-client authentication issue, not a Valkey TLS or server failure. Phase 8C.6 already proved authenticated TLS PONG using the explicit `-a` option. The replication attempt stopped at Step 1, so `ulc-01/02/03` remain in the Phase 8C.6 standalone TLS-only authenticated state and no `replicaof` directive has yet been applied.

Correction: all remaining commissioning commands use explicit `-a "$AUTH"` for `valkey-cli` rather than relying on the environment-variable path. The secret remains unprinted and root-only. Valkey's upstream CLI documentation supports both `-a` and `VALKEYCLI_AUTH`, but actual observed package behavior is authoritative for this deployment.

The corrected Phase 8C.7 replication deployment then passed.

Final topology:

```
ulc-01 10.104.0.2 = PRIMARY
ulc-02 10.104.0.4 = REPLICa
ulc-03 10.104.0.8 = REPLICa
```

Observed primary state:

```
role:master
connected_slaves:2
slave0:ip=10.104.0.4,port=6379,state=online,offset=14,lag=0
slave1:ip=10.104.0.8,port=6379,state=online,offset=14,lag=0
```

Both replicas reported `role:slave`, `master_host:10.104.0.2`, `master_port:6379`, and `master_link_status:up`. Primary write propagation passed to both replicas, and both replicas correctly rejected direct writes with `READONLY`.

Result:

```
Phase 8C.7 Valkey TLS replication: PASS
```

8.21.10 Phase 8C.8 Sentinel foundation precheck checkpoint

The Sentinel foundation precheck has started. `ulc-01` completed successfully and showed:

```
no separate Sentinel package installed
valkey-sentinel package candidate: 7.2.13+dfsg1-0ubuntu0.1
redis-sentinel package candidate also exists but is not selected
/usr/bin/valkey-server present
no Sentinel systemd unit currently installed
valkey-server supports Sentinel mode via --sentinel
port 26379 free
```

This confirms the correct family is the matching `valkey-sentinel 7.2.13` package, not the older Redis Sentinel package.

The first continuation attempt reached `ulc-02` and confirmed:

```
valkey-sentinel candidate: 7.2.13+dfsg1-0ubuntu0.1
valkey-sentinel not installed
/usr/bin/valkey-server present
valkey-server supports Sentinel mode via --sentinel
```

A second continuation completed the remaining ulc-02 checks:

```
no Sentinel systemd unit
26379 free
```

ulc-02 therefore matches the validated ulc-01 Sentinel foundation baseline: matching valkey-sentinel 7.2.13 candidate, package not installed, Sentinel mode supported by valkey-server, no Sentinel unit yet, and port 26379 free.

The ulc-03 precheck then confirmed the host, matching valkey-sentinel candidate 7.2.13+dfsg1-0ubuntu0.1, package-not-installed state, /usr/bin/valkey-server, and visible Sentinel-mode support. The script stopped immediately after printing the Sentinel-mode help text, before the explicit PASS line, systemd-unit check, and port 26379 check.

The next ulc-03 continuation also stopped at the Sentinel capability step before any package/config/service mutation. The command used `HELP="$(valkey-server --help 2>&1)"` while the wrapper was running with `set -e`. On this Ubuntu Valkey build, valkey-server --help prints valid help text but exits non-zero, so the command substitution assignment itself triggers `set -e` and terminates the script before the captured text can be checked. The visible earlier output already proves that the help text contains `Sentinel mode;`; this is a shell-control-flow issue, not a Sentinel capability failure.

The corrected ulc-03 continuation passed. Final Phase 8C.8 Sentinel foundation precheck state:

```
ulc-01: PASS
ulc-02: PASS
ulc-03: PASS
valkey-sentinel candidate = 7.2.13+dfsg1-0ubuntu0.1
Sentinel package not installed
valkey-server supports --sentinel
no Sentinel systemd unit
26379 free
```

Result:

```
Phase 8C.8 Sentinel foundation precheck: PASS
```

Phase 8C.9 installed and inspected the matching valkey-sentinel package on all three nodes. Observed package version on ulc-01, ulc-02, and ulc-03: 7.2.13+dfsg1-0ubuntu0.1.

Package/runtime evidence:

```
executable: /usr/bin/valkey-sentinel
service: /usr/lib/systemd/system/valkey-sentinel.service
ExecStart: /usr/bin/valkey-sentinel /etc/valkey/sentinel.conf --supervised systemd --daemonize no
User: valkey
Group: valkey
configuration: /etc/valkey/sentinel.conf
configuration permissions: 640 valkey:valkey
RuntimeDirectory: sentinel
ReadWriteDirectories: /etc/valkey
```

The packaged default configuration is not suitable for production as-is:

```
protected-mode no
port 26379
sentinel monitor mymaster 127.0.0.1 6379 2
```

Immediately after package installation, valkey-sentinel was stopped and disabled on every node. Final safety verification passed on all three nodes:

```
ActiveState=inactive
UnitFileState=disabled
26379 closed: PASS
```

Result:

```
Phase 8C.9 Sentinel package foundation: PASS
```

No Sentinel quorum is active yet. Do not start or enable the service with the packaged default configuration.

Phase 8C.10 Sentinel TLS configuration preflight passed on ulc-03. Sentinel remained inactive and disabled, and port 26379 remained closed. The packaged Sentinel configuration exposes only the default plaintext port 26379 directive and the default sentinel monitor mymaster 127.0.0.1 6379 2; no TLS directives are pre-populated in /etc/valkey/sentinel.conf.

The valkey service account can read all staged TLS files (ca.crt, server.crt, and server.key, each 640 root:valkey). The current Valkey topology was also reconfirmed before Sentinel configuration:

```
role:master
connected_slaves:2
```

The existing writable primary remains 10.104.0.2:6379 with both replicas connected.

Valkey's TLS documentation confirms that Sentinel inherits the common TLS directives and that tls-replication yes controls both Sentinel-to-Valkey TLS and TLS on Sentinel's own inter-Sentinel listener. Therefore the production Sentinel configuration will explicitly disable plaintext with port 0, enable tls-port 26379, set the staged certificate/key/CA paths, set tls-replication yes, and retain quorum 2.

Result:

```
Phase 8C.10 Sentinel TLS configuration preflight: PASS
```

Phase 8C.11 TLS-only Sentinel quorum deployment progressed successfully through quorum validation. All three Sentinels started with node-specific private binds, plaintext disabled (port 0), TLS-only tls-port 26379, the staged certificate/key/CA files, tls-auth-clients no, tls-replication yes, separate Sentinel password authentication, and monitoring of lorawan-valkey at the current primary 10.104.0.2:6379 with quorum 2.

Validated runtime evidence:

```
ulc-01 Sentinel: active, TLS listener 127.0.0.1:26379 + 10.104.0.2:26379
ulc-02 Sentinel: active, TLS listener 127.0.0.1:26379 + 10.104.0.4:26379
ulc-03 Sentinel: active, TLS listener 127.0.0.1:26379 + 10.104.0.8:26379
TLSv1.3 handshake to all three: PASS
CA verification: OK
peer name valkey.internal.lorawan.com: verified
all three Sentinels report primary 10.104.0.2:6379
two Sentinel peers discovered: PASS
CKQUORUM: OK 3 usable Sentinels. Quorum and failover authorization can be reached
```

The enablement continuation then confirmed ulc-03 was successfully enabled and remained active. Final service-state verification also passed on ulc-01 and ulc-02:

```
ulc-03 service=active, enabled=enabled
ulc-01 service=active, enabled=enabled, listeners 127.0.0.1:26379 + 10.104.0.2:26379
ulc-02 service=active, enabled=enabled, listeners 127.0.0.1:26379 + 10.104.0.4:26379
```

The final Sentinel verification continuation passed completely. ulc-03 exposes the expected TLS-only listeners on 127.0.0.1:26379 and 10.104.0.8:26379; all three Sentinel members report the same current primary 10.104.0.2:6379; ulc-03 sees exactly two Sentinel peers; and SENTINEL CKQUORUM lorawan-valkey returns OK 3 usable Sentinels. Quorum and failover authorization can be reached.

Final Phase 8C.11 state:

```
ulc-01 Sentinel = active + enabled
ulc-02 Sentinel = active + enabled
ulc-03 Sentinel = active + enabled
Sentinel transport = TLS-only :26379
Sentinel members = 3
quorum = 2
current primary = 10.104.0.2:6379
peer_count = 2
primary discovery = PASS on all three members
CKQUORUM = PASS
```

Result:

```
Phase 8C.11 TLS Sentinel quorum: PASS
```

Phase 8C.12 controlled automatic failover testing started. The starting-state check passed with Sentinel reporting 10.104.0.2:6379 as the primary, CKQUORUM returned OK 3 usable Sentinels, and the pre-failover test write to ulc-01 succeeded.

The script then stopped immediately at Step 4 while attempting to run `sudo systemctl stop valkey-server` remotely on ulc-01 over SSH. No output confirmed that the service actually stopped, and no automatic promotion output followed. Therefore automatic failover has **not yet been proven** and the test must continue from a verified current-state check rather than assuming ulc-01 went down.

This matches the earlier commissioning pattern where remote `sudo` from ulc-03 can terminate a non-interactive SSH block when `sudo` requires a terminal/password path. The correction is to first inspect ulc-01's actual Valkey service state and Sentinel's current master view. If ulc-01 is still active, stop it with an SSH allocation that permits the remote `sudo` interaction, while leaving `valkey-sentinel` running. If it is already inactive, do not stop it again. Then continue observing Sentinel without manually changing replication.

The corrected Phase 8C.12 continuation immediately observed that Sentinel's current primary had already changed to 10.104.0.8 (ulc-03):

```
=== PHASE 8C.12 AUTOMATIC FAILOVER CONTINUATION ===
=== STEP 1 - DISCOVER CURRENT STATE ===
Sentinel current primary = 10.104.0.8
```

This is strong evidence that the previous failure injection actually succeeded before the shell returned: ulc-01 stopped long enough for Sentinel to complete an automatic election and promote ulc-03. Do not stop any Valkey node again and do not manually issue `replicaof`. The remaining work is recovery verification, not another failover trigger.

Phase 8C.12 recovery verification then advanced substantially. All three Sentinels agreed on ulc-03 (10.104.0.8) as the elected primary. ulc-03 reported `role:master` with ulc-02 already online as a replica. The original pre-failover test key `phase8c12:failover:1787621601` was still present on ulc-03 with its expected `before-*` value, proving the pre-failover write survived the automatic promotion. A fresh authenticated TLS write to ulc-03 also succeeded.

Observed checkpoint:

```
ulc-01 Sentinel reports primary=10.104.0.8
ulc-02 Sentinel reports primary=10.104.0.8
ulc-03 Sentinel reports primary=10.104.0.8
three-Sentinel consensus: PASS
ulc-03 role:master
connected_slaves:1
ulc-02 online as replica
pre-failover data survived: PASS
new-primary write: PASS
```

The pasted output then ended at STEP 6 - INSPECT OLD PRIMARY ULC-01 before the ulc-01 service-state result was printed. A later minimized recovery block also stopped immediately at STEP 1 - CHECK ULC-01 SSH + VALKEY STATE with no remote output. This means the remaining problem is specifically the ulc-03 -> ulc-01 SSH inspection path, not the already-proven Sentinel failover. Do not trigger another failover and do not modify replication manually.

Phase 8C.12 is now proven through automatic promotion, Sentinel consensus, data survival, and successful writes on the promoted primary.

The follow-up SSH diagnostic to ulc-01 also completed successfully. SSH authentication with `/root/.ssh/cloud-deployment-phase8` is healthy and the remote host key matches. The remote service state is:

```
host=ulc-01
valkey=inactive
sentinel=active
ssh exit status=0
```

This confirms the original failure injection succeeded exactly as intended: only `valkey-server` on ulc-01 is down, while the local Sentinel stayed alive and participated in the election. There is no SSH or host reachability problem.

The recovery step then successfully started `valkey-server` on ulc-01. Immediate service verification showed:

```
valkey=active
sentinel=active
```

No replication directive was changed manually and ulc-03 remains the Sentinel-elected primary.

The final read-only recovery verification is currently waiting for ulc-01 to be reconfigured by Sentinel after its service restart. The observed output has reached at least attempt 18 of the bounded 60-attempt loop without yet matching all required replica fields. This is not yet a failure because the script intentionally allows up to 120 seconds (60 attempts at 2-second intervals) for recovery convergence.

The bounded rejoin wait did not immediately converge, so a read-only Sentinel diagnostic was captured. Sentinel still reports ulc-03 (10.104.0.8:6379) as the healthy master, with `num-slaves=2`, `num-other-sentinels=2`, quorum 2, and failover epoch 1.

Sentinel's replica view now proves that ulc-01 has been logically converted to a replica of ulc-03:

```
ulc-02 10.104.0.4:6379
flags=slave
master-host=10.104.0.8
master-port=6379
master-link-status=ok
```

```
ulc-01 10.104.0.2:6379
flags=slave
master-host=10.104.0.8
master-port=6379
master-link-status=err
```

The ulc-01 Sentinel log confirms the automatic recovery action itself occurred:

```
-sdown slave 10.104.0.2:6379 ... @ lorawan-valkey 10.104.0.8 6379
+convert-to-slave slave 10.104.0.2:6379 ... @ lorawan-valkey 10.104.0.8 6379
```

Therefore Sentinel failover and automatic topology rewriting are proven. The only remaining recovery defect is the data-plane replication link from ulc-01 to the elected primary: Sentinel sees ulc-01 as a slave of 10.104.0.8, but reports master-link-status=err.

The first read-only replication-link diagnostic captured the following ulc-01 state:

```
role:slave
master_host:10.104.0.8
master_port:6379
master_link_status:down
master_last_io_seconds_ago:-1
master_sync_in_progress:1
master_sync_total_bytes:-1
master_sync_read_bytes:0
master_sync_left_bytes:-1
connected_slaves:0
```

This proves ulc-01 has accepted the Sentinel-directed replica role and is attempting an initial/full synchronization, but the data connection has not progressed far enough to receive any bytes. The diagnostic then stopped at the configuration-inspection step because opsadmin cannot read /etc/valkey/valkey.conf directly:

```
grep: /etc/valkey/valkey.conf: Permission denied
```

That permission error is from the diagnostic command only; it is not evidence of a Valkey runtime failure. The configuration file is intentionally restricted.

The corrected privileged config read then exposed an important clue on ulc-01:

```
replica-read-only yes
bind 127.0.0.1 10.104.0.2
port 0
tls-port 6379
tls-cert-file /etc/lorawan-pki/valkey/server.crt
tls-key-file /etc/lorawan-pki/valkey/server.key
tls-ca-cert-file /etc/lorawan-pki/valkey/ca.crt
tls-auth-clients no
tls-replication yes
replicaof 10.104.0.8 6379
```

masterauth is notably absent from the active replication/security directives even though the Valkey primary requires password authentication. That is a strong candidate explanation for the stalled full sync: Sentinel correctly rewrote replicaof to the newly elected primary, but the restarted former primary may no longer possess the credential needed for replica-to-primary authentication.

The direct TLS test in the same block failed only because it was run as opsadmin, which cannot read /etc/lorawan-pki/valkey/ca.crt (root:valkey 640). TCP reachability to 10.104.0.8:6379 passed, so that CA permission error was a diagnostic-user artifact, not proof of a TLS transport failure.

The follow-up confirmation then proved the transport and primary credentials are healthy:

```
masterauth/masteruser absent from /etc/valkey/valkey.conf
TLSv1.3 handshake from ulc-01 to 10.104.0.8:6379: PASS
CA verification: OK
Verified peername: valkey.internal.lorawan.com
authenticated valkey-cli PING from ulc-01 to ulc-03: PONG
ulc-01 role:slave
master_host:10.104.0.8
master_port:6379
master_link_status:down
```

The attempted runtime `CONFIG GET masterauth` returned `NOAUTH Authentication required` because that local diagnostic command itself did not authenticate, so it does not establish the runtime value. However, the persistent configuration definitely lacks `masterauth`, while a manually authenticated TLS client from the same host reaches the elected primary successfully. Combined with the stalled replica link, this isolates the commissioning defect to the missing replica authentication credential rather than network, CA, hostname verification, or primary availability.

Root cause: Phase 8C.7 placed `masterauth` only on the nodes that were replicas at that moment. `ulc-01` was the original primary and therefore did not receive a persistent `masterauth`. After automatic failover, Sentinel correctly changed `ulc-01` to follow `ulc-03`, but Sentinel's topology rewrite does not supply the persistent Valkey replication password. For failover-safe symmetry, every Valkey node must retain the same `masterauth` even while it is primary; the directive is inert while a node is primary and becomes necessary if that node later becomes a replica.

The `masterauth` repair was then applied successfully on `ulc-01`. A timestamped backup was created at `/etc/valkey/valkey.conf.phase8c12-masterauth-before-20260825-020750`; the Sentinel-selected `replicaof 10.104.0.8 6379` directive was explicitly verified and preserved; exactly one persistent `masterauth` directive was written; `CONFIG SET masterauth` returned `OK`; and an authenticated `CONFIG GET masterauth` matched the expected secret without printing it.

Observed repair evidence:

```
replicaof 10.104.0.8 6379
masterauth lines=1
persistent masterauth: PASS
runtime masterauth: PASS
runtime masterauth verification: PASS
ulc-01 masterauth repair complete
```

The combined automation then dropped into an interactive root shell on `ulc-01` instead of returning cleanly to the outer `ulc-03` script, so the remaining verification was rerun as a separate continuation from `ulc-03`. This was an orchestration/TTY here-document issue, not a repair failure.

The post-repair verification immediately confirmed that the missing `masterauth` was the actual recovery defect. On the first check after applying the credential, `ulc-01` reported:

```
attempt=1 link=up sync=0
role:slave
master_host:10.104.0.8
master_port:6379
master_link_status:up
master_last_io_seconds_ago:1
master_sync_in_progress:0
ulc-01 replication recovery: PASS
ulc-01 persistent masterauth: PASS
```

This is direct causal evidence: no `replicaof` change, service restart, or fallback was performed; adding the correct persistent/runtime `masterauth` alone restored the replica link to the Sentinel-elected primary.

The next all-node symmetry audit then failed on `ulc-02`:

```
FAIL: ulc-02 masterauth mismatch
```

Therefore Phase 8C.12 recovery is not yet fully closed. ulc-01 is healthy again, but failover-safe authentication is not normalized across all three Valkey nodes. The correct design is for every node to retain the same masterauth even when currently primary, because any node can later be converted into a replica by Sentinel.

The final failover-safe masterauth normalization then completed successfully on ulc-02 and ulc-03 without restarting Valkey and without altering Sentinel-selected topology. ulc-02 preserved replicaof 10.104.0.8 6379, received exactly one persistent masterauth, accepted the same value at runtime, and passed secret-equality verification without printing the credential. ulc-03 remained role:master, received the same persistent/runtime masterauth, and therefore all three nodes now carry the credential required if Sentinel later converts any current or former primary into a replica.

Final recovered topology:

```
ulc-03 10.104.0.8 = PRIMARY
ulc-01 10.104.0.2 = REPLICAS -> 10.104.0.8:6379, master_link_status:up
ulc-02 10.104.0.4 = REPLICAS -> 10.104.0.8:6379, master_link_status:up
ulc-03 connected_slaves:2
```

Final replication evidence:

```
slave0:ip=10.104.0.4,port=6379,state=online,offset=623818,lag=0
slave1:ip=10.104.0.2,port=6379,state=online,offset=623536,lag=0
primary write: PASS
ulc-01 write propagation: PASS
ulc-02 write propagation: PASS
```

All three Sentinels continued to report 10.104.0.8 as the primary, and SENTINEL CKQUORUM lorawan-valkey returned:

```
OK 3 usable Sentinels. Quorum and failover authorization can be reached
```

Phase 8C.12 is therefore fully closed:

```
Automatic Sentinel promotion = PASS
Pre-failover data survival = PASS
New-primary writes = PASS
ulc-01 masterauth repair = PASS
ulc-01 replication recovery = PASS
ulc-02 masterauth normalization = PASS
ulc-03 masterauth normalization = PASS
Failover-safe masterauth on every node = PASS
One primary + two replicas = PASS
Final write propagation = PASS
Three-Sentinel consensus = PASS
Quorum after recovery = PASS
```

Result:

```
Phase 8C.12 automatic failover + recovery: PASS
```

Keep ulc-03 as the Sentinel-elected primary. Do not manually fail back to ulc-01.

Next: Phase 8C.13 HAProxy writable-primary routing. Before mutation, preflight the existing HAProxy 2.8 deployment on the application nodes, confirm port 16379 is free, confirm the HAProxy service identity and CA-read path, and validate the exact TLS health-check syntax against the installed HAProxy build. The Valkey node

certificates contain the shared SAN `valkey.internal.lorawan.com`; do not use an unissued `valkey-ha.internal.lorawan.com` name for backend certificate verification.

The first Phase 8C.13 read-only preflight reconfirmed the Valkey/Sentinel baseline before inspecting HAProxy:

```
ulc-03 role:master
connected_slaves:2
ulc-01 + ulc-02 replicas online
Sentinel primary: 10.104.0.8:6379
Valkey topology: PASS
Sentinel primary: PASS
```

The script then reached `ulc-01` successfully but stopped at the HAProxy package-version command with:

```
bash: line 12: Status: unbound variable
```

Cause: the remote shell was running `set -u` and the `dpkg-query -f='${Status}|${Version}\n'` format string used double quotes. Bash attempted to expand `${Status}` and `${Version}` as shell variables before `dpkg-query` could interpret them, and the unset `Status` variable terminated the preflight. No HAProxy configuration, listener, or service state was changed.

Correction: quote the `dpkg-query` format with single quotes (`-f='${Status}|${Version}\n'`) inside the remote shell. Resume the Phase 8C.13 preflight from the HAProxy-node inspection; there is no need to repeat the already-passed Valkey/Sentinel baseline unless topology changes before the retry.

The corrected continuation then advanced through `ulc-01` and exposed the next concrete preflight issue. HAProxy is the expected pinned build `2.8.16-0ubuntu0.24.04.3`, active and enabled, with existing PostgreSQL listeners `10.104.0.2:15432/15433`, MQTT frontend `10.104.0.2:8883`, and Mosquitto backend listener `8884`; target port `16379` is free. `/etc/haproxy/haproxy.cfg` is `0644 root:root` and the expected existing PostgreSQL/MQTT sections remain present.

The Valkey CA cannot be traversed by the HAProxy service account because `/etc/lorawan-pki/valkey` is `0750 root:valkey`; `namei` therefore ends with `ca.crt - Permission denied for non-valkey users`. Do **not** add `haproxy` to the `valkey` group, because that group can also read the Valkey server private key (`0640 root:valkey`). Instead, install a CA-only copy for HAProxy under a dedicated HAProxy-readable path such as `/etc/haproxy/ca/valkey-ca.crt`, with no server certificate or private key copied. Validate certificate equality by SHA-256 before use.

The CA-only HAProxy trust-path correction then completed successfully on both application nodes. The commissioned Valkey CA hash on `ulc-03` is `6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb`, and the copied CA on both `ulc-01` and `ulc-02` matched this SHA-256 exactly.

Installed trust boundary on both HAProxy nodes:

```
/etc/haproxy/ca          750 root:haproxy
/etc/haproxy/ca/valkey-ca.crt 640 root:haproxy
HAProxy CA read: PASS
Valkey private key remains inaccessible to HAProxy: PASS
CA equality: PASS
```

The original `/etc/lorawan-pki/valkey` permissions were not weakened and the HAProxy account was not added to the `valkey` group. This preserves separation between CA trust and private-key access.

The remaining HAProxy preflight evidence passed on both `ulc-01` and `ulc-02`:

```
haproxy 2.8.16-0ubuntu0.24.04.3
service active + enabled
16379 free
```

```
existing PostgreSQL/MQTT listeners preserved
current /etc/haproxy/haproxy.cfg syntax valid
HAProxy dedicated CA readable
10.104.0.2:6379 reachable
10.104.0.4:6379 reachable
10.104.0.8:6379 reachable
```

The pasted execution ended after the ulc-02 TCP-backend reachability checks, before the separate all-backend TLS hostname-verification section printed. Therefore the CA trust staging and TCP preflight are accepted, but Phase 8C.13 is not yet closed.

The TLS identity and current-role portion of the next Phase 8C.13 preflight passed completely. All three Valkey backends negotiated TLS 1.3 with TLS_AES_256_GCM_SHA384, verified against the commissioned CA, and matched the shared certificate name `valkey.internal.lorawan.com`. Runtime role checks reconfirmed ulc-01 and ulc-02 as replicas following 10.104.0.8:6379 with links up, and ulc-03 as `role:master` with two connected replicas.

The first offline HAProxy TLS-backend syntax test failed before any production mutation. The temporary test config split one server directive across multiple physical lines without continuation syntax:

```
server test-primary
  10.104.0.8:6379
  check
  ssl
  verify required
  ca-file /etc/haproxy/ca/valkey-ca.crt
  sni str(valkey.internal.lorawan.com)
```

HAProxy 2.8 therefore parsed each subsequent line as a separate backend keyword and reported unknown keyword `'10.104.0.8:6379'`, followed by equivalent errors for `check`, `ssl`, `verify`, `ca-file`, and `sni`. This is a configuration-formatting error, not rejection of TLS server parameters. No production HAProxy config was changed or reloaded.

Correction: keep the full server directive on one logical line (or use explicit shell-generated single-line output), then validate the TLS backend syntax offline. After that, validate the exact `tcp-check connect/send/expect` syntax separately before introducing any health-check credential.

The corrected single-line server syntax was accepted by HAProxy, but the temporary test file contained only a backend and no frontend/listen section. `haproxy -c` therefore printed:

```
Configuration file has no error but will not start (no listener) => exit(2).
```

This is not a syntax failure. It explicitly confirms that HAProxy found no configuration error; exit code 2 is produced because the isolated test file has no listener and therefore would not start as a runtime configuration. No production config was changed or reloaded.

The corrected offline validation then passed on both ulc-01 and ulc-02 by adding a harmless dummy loopback frontend only inside the temporary test configuration. No listener was actually started because validation still used `haproxy -c` only.

Validated on both HAProxy nodes:

```
TLS backend server-line syntax: PASS
tcp-check TLS connect syntax: PASS
tcp-check send/expect syntax: PASS
combined three-backend TLS health-check grammar: PASS
production /etc/haproxy/haproxy.cfg remains valid: PASS
```

This proves HAProxy 2.8.16 accepts the required backend TLS, CA verification, SNI, tcp-check connect, tcp-check send, and tcp-check expect grammar. The production HAProxy configuration remains untouched and no reload/restart occurred.

Result:

```
Phase 8C.13 HAProxy offline TLS/tcp-check syntax validation: PASS
```

The dedicated least-privilege HAProxy health-check ACL identity was then deployed successfully on all three Valkey nodes.

Identity:

```
username = haproxy-health
password = separate randomly generated health-check credential
credential storage = /root/lorawan-secrets/valkey-haproxy-health.txt on ulc-03 during commissioning
```

The ACL is persisted symmetrically in /etc/valkey/valkey.conf on ulc-01, ulc-02, and ulc-03 and was also applied at runtime with ACL SETUSER, without restarting Valkey or changing replication topology.

Effective ACL policy:

```
on
-@all
+ping
+info
```

Observed validation on every node:

```
PING -> PASS
INFO replication -> PASS
GET -> NOPERM / denied
SET -> NOPERM / denied
```

Role detection through the dedicated account also matched the commissioned topology:

```
ulc-01 role=slave
ulc-02 role=slave
ulc-03 role=master
least-privilege role detection: PASS
```

Backups were created before each persistent ACL change. The main Valkey application/replication password was not reused for HAProxy health checks and no production HAProxy configuration was changed.

Result:

```
Phase 8C.13 dedicated HAProxy health-check user: PASS
```

The dedicated HAProxy health credential was then staged successfully on both HAProxy nodes under /etc/haproxy/secrets/valkey-health.env, with directory 0750 root:haproxy and file 0640 root:haproxy. The haproxy service account can read this dedicated credential and /etc/haproxy/ca/valkey-ca.crt, while /etc/lorawan-pki/valkey/server.key remains inaccessible. Secret equality between ulc-03's root-only source and both HAProxy copies was verified by SHA-256 comparison without printing the credential.

Observed checkpoint:

```
ulc-01 health credential access: PASS
ulc-01 CA access: PASS
ulc-01 Valkey private-key isolation: PASS
ulc-01 credential equality: PASS
ulc-02 health credential access: PASS
ulc-02 CA access: PASS
ulc-02 Valkey private-key isolation: PASS
ulc-02 credential equality: PASS
```

The direct authenticated role queries and offline HAProxy role-check validation then completed successfully from both HAProxy nodes. Using only the dedicated haproxy-health identity, both ulc-01 and ulc-02 independently observed the same Valkey topology:

```
ulc-01 role=slave
ulc-02 role=slave
ulc-03 role=master
```

The complete HAProxy 2.8 check grammar also validated offline on both nodes:

```
TLS to backend + CA verification
AUTH haproxy-health <dedicated credential>
expect +OK
INFO replication
expect role:master
```

The credential was supplied through VALKEY_HAPROXY_HEALTH_PASSWORD and was verified absent from the temporary HAProxy config text. The production /etc/haproxy/haproxy.cfg remained valid and unchanged, port 16379 remained free on both HAProxy nodes, and no reload/restart occurred.

Result:

```
Phase 8C.13 authenticated HAProxy role-check preflight: PASS
```

Production design decision: the :16379 frontend will remain TCP/TLS passthrough for application traffic, so client TLS terminates at the currently selected Valkey node and the shared certificate identity valkey.internal.lorawan.com continues to be verified end-to-end. Do not add ssl to the production backend server lines, because that would wrap an already TLS-speaking client stream in a second TLS layer. The health check itself will still use tcp-check connect ssl with CA verification and SNI. A systemd EnvironmentFile=/etc/haproxy/secrets/valkey-health.env drop-in is required so HAProxy can resolve % [env(VALKEY_HAPROXY_HEALTH_PASSWORD)] without storing the secret in haproxy.cfg. Because this environment must be present in the HAProxy master process, commission the two HAProxy nodes sequentially and restart each one only after validating its candidate config, leaving the other HAProxy node available during the brief restart.

The first production :16379 deployment was then attempted on ulc-01 only. Safety checks, backup creation, candidate generation, systemd EnvironmentFile integration, candidate validation, HAProxy restart, service state, master-process environment inheritance, production config validation, and the new 10.104.0.2:16379 listener all passed. Existing PostgreSQL (15432/15433) and MQTT (8883) listeners remained present.

The first functional test exposed a routing defect:

```
PING=PONG
routed role=slave
FAIL: ulc-01 HAProxy did not route to writable primary
```

So TLS passthrough itself is working, but the production health-check behavior is not yet excluding replicas. ulc-02 has NOT yet received the production :16379 configuration and must remain untouched until ulc-01 is corrected.

Current safe state:

```
ulc-01 :16379 listener = active
ulc-01 TLS passthrough = PASS
ulc-01 writable-primary selection = FAIL (routed to replica)
ulc-02 production :16379 deployment = NOT STARTED
```

The exact production block on ulc-01 was then inspected and matched the intended design: frontend `valkey_primary` binds `10.104.0.2:16379`; the backend uses option `tcp-check`; TLS health connections use SNI `valkey.internal.lorawan.com`; AUTH uses the dedicated environment-backed `haproxy-health` credential; INFO replication is followed by expect string `role:master`; and all three Valkey nodes are listed as checked backends. The production block therefore contains no obvious backend-order or plaintext/TLS configuration mistake.

A likely remaining explanation was HAProxy initial server state: checked servers may initially be available until failed health checks mark replicas DOWN. A delayed read-only re-test was therefore started after confirming the real Valkey roles remained `ulc-01=slave`, `ulc-02=slave`, `ulc-03=master` and waiting 15 seconds for health checks to settle.

The delayed test then terminated at the very first `:16379` role query before printing any routed role. Because the wrapper used `set -e` and the `valkey-cli` call was inside command substitution, any non-zero client result aborted the script before the HAProxy log section could run. This new evidence means the endpoint may have transitioned from the initial unsafe state (replica considered eligible) to a state with no eligible backend at all. That would point to a runtime health-check failure affecting every backend, including the real primary, rather than only an initial-state window. This must be diagnosed explicitly; do not assume `init-state fully-down` is the fix yet.

The first runtime diagnostic checkpoint confirmed that HAProxy itself remains healthy and the production listener still exists after the role checks have had time to settle:

```
haproxy=active
LISTEN 10.104.0.2:16379
```

The smaller runtime diagnostic then identified the actual health-check failure. `10.104.0.2:16379` remained listening and HAProxy remained active, but the client received `SSL_connect failed: unexpected eof while reading`. HAProxy logs showed all three Valkey backends becoming DOWN at exactly the same point: `Layer7 timeout ... at step 3 of tcp-check (expect string '+OK')`. After that HAProxy reported backend `'valkey_primary_backend'` has no server available, and later frontend traffic was logged against `<NOSRV>`. Therefore the failure occurs before INFO replication and before the `role:master` matcher; HAProxy is not receiving the `+OK` response to its AUTH command.

The production AUTH rule currently uses `tcp-check send-lf "AUTH haproxy-health %[env{VALKEY_HAPROXY_HEALTH_PASSWORD}]\r"`. HAProxy's documented Redis role-check examples use exact protocol sends ending in `\r\n`, such as `tcp-check send info\ replication\r\n`. The next safe proof is a temporary loopback-only HAProxy instance on ulc-01 using the same dedicated credential but rendering an exact static `tcp-check send AUTH\ haproxy-health\ <credential>\r\n` into a mode-0600 temporary config. This must not touch `/etc/haproxy/haproxy.cfg` or the systemd service. If that temporary check marks the replicas DOWN at `role:master` and ulc-03 UP, the root cause is the production `send-lf AUTH` framing rather than TLS, ACL permissions, or role detection. Destroy the temporary config/process after the test, keep ulc-02 production deployment untouched, and only then correct the production check.

A temporary loopback-only HAProxy proof was then run on ulc-01 using exact CRLF sends for both AUTH and INFO replication. The temporary configuration validated successfully, started successfully, and was protected mode 0600. The first functional request was issued only about five seconds after startup and still returned a replica (`role:slave`). This result does not yet disprove the exact-CRLF correction because the temporary backend used `timeout check 6s`: a replica can remain initially eligible while its first `role:master` expectation is still waiting to time

out. The proof therefore ended too early to distinguish framing failure from the initial health-check convergence window.

The settled exact-CRLF proof then passed. After 12 seconds, ulc-01 and ulc-02 were both marked DOWN specifically at step 5, expect string 'role:master', while ulc-03 remained eligible. Ten consecutive requests through the temporary loopback frontend all returned role:master, with master_routes=10, non_master_routes=0, and errors=0. This confirms the health-check protocol itself works when AUTH and INFO replication are sent with exact CRLF framing. It also confirms the initial unsafe replica routing was caused by the health-check convergence window: checked backends were initially eligible before their first role check completed.

Production correction definitely requires replacing the broken AUTH/INFO framing with exact CRLF and using a bounded role matcher. The isolated min-recv proof then passed on HAProxy 2.8.16: tcp-check expect min-recv 64 string role:master marked ulc-01 DOWN in 5 ms and ulc-02 DOWN in 14 ms with Layer7 invalid response, while ulc-03 remained eligible. After only a two-second wait, ten consecutive requests returned role:master, with master_routes=10, non_master_routes=0, and errors=0. This eliminates the earlier six-second convergence delay for replicas.

Validated HAProxy 2.8 production behavior is therefore: TLS health connection with SNI and CA verification, exact CRLF for AUTH and INFO replication, min-recv 64 on the role:master expectation, and the dedicated haproxy-health ACL identity. Do not use init-state fully-down on these 2.8 nodes. Apply the correction to ulc-01 only first, validate and reload HAProxy, wait briefly for checks to converge, and require repeated master-only routing before deploying the same corrected block to ulc-02.

The first ulc-01 production correction attempt did not reach validation or reload. Its line-aware Python rewrite succeeded in locating all three target rules, but it wrote the HAProxy escape sequences as doubled backslashes (\\r\\n) rather than the required single escaped CRLF tokens (\r\n). The immediately following literal assertions therefore failed and the wrapper exited before haproxy -c or systemctl reload haproxy. A second normalization pass corrected only those two send-lf lines, preserved min-recv 64, and passed exact-line assertions before reload.

The final ulc-01 production correction is now validated and live. The production rules are tcp-check send-lf "AUTH haproxy-health %[env(VALKEY_HAPROXY_HEALTH_PASSWORD)]\r\n", tcp-check send-lf "INFO replication\r\n", and tcp-check expect min-recv 64 string role:master. haproxy -c passed, HAProxy reloaded successfully, and the existing PostgreSQL 15432/15433, MQTT 8883, and Valkey 16379 listeners all remained present. Health logs marked ulc-01 and ulc-02 DOWN at the role:master matcher in 6 ms and 5 ms respectively, while ulc-03 remained eligible. Ten consecutive requests through 10.104.0.2:16379 all returned role:master (master_routes=10, non_master_routes=0, errors=0). A write through the ulc-01 HAProxy endpoint succeeded and the value was verified directly on the current ulc-03 primary, then the test key was removed. Therefore ulc-01 production writable-primary routing is PASS.

The same proven writable-primary configuration is now deployed on ulc-02 at 10.104.0.4:16379. The dedicated health credential and CA were already present and readable only by the HAProxy service as intended; the Valkey private key remained inaccessible. The candidate configuration passed exact AUTH/INFO CRLF assertions, min-recv 64, secret-exclusion, and haproxy -c validation. The systemd EnvironmentFile drop-in was installed, HAProxy restarted successfully so the master process inherited the health credential, and existing PostgreSQL 15432/15433 plus local Mosquitto 8884 listeners remained present. Health logs rejected ulc-01 and ulc-02 as replicas in 7 ms and 6 ms respectively. Ten consecutive requests through 10.104.0.4:16379 all returned role:master with zero errors. Both HAProxy endpoints then independently reported role:master; a write through ulc-02 was read successfully through ulc-01 and verified directly on ulc-03 before cleanup. Phase 8C.13 dual HAProxy writable-primary routing is PASS.

Phase 8C.14 completed the end-to-end automatic failover proof. The starting primary was ulc-03 (10.104.0.8) with healthy Sentinel quorum and both HAProxy :16379 endpoints routing to the writable master. A pre-failover key was written through ulc-01 HAProxy and read through ulc-02. Only the ulc-03 Valkey service was then stopped; its Sentinel remained active. Sentinel elected ulc-02 (10.104.0.4) as the new primary after seven polling attempts,

and all three Sentinels agreed on that new primary. Direct verification showed ulc-02 as `role:master` with ulc-01 online as a replica.

Both HAProxy endpoints automatically followed the promoted primary with no HAProxy configuration change or reload. Each endpoint was briefly unavailable for one polling attempt during convergence, then routed to `role:master`; ten consecutive post-failover requests through each endpoint were master-only with zero errors. The pre-failover value survived, a new write through ulc-02 HAProxy succeeded, the new value was read through ulc-01 HAProxy, and it was verified directly on the new ulc-02 primary. ulc-03 Valkey was then restarted; it transitioned from its prior master state to `role:slave`, followed ulc-02, and reached `master_link_status:up` on attempt 9. Final topology was ulc-02 primary with ulc-01 and ulc-03 online as replicas (`connected_slaves:2`). Sentinel quorum remained healthy, the test key was removed, and no manual failback was performed.

Phase 8C Valkey HA is therefore complete: TLS-only Valkey on three nodes, authenticated replication, three TLS Sentinels with quorum 2, automatic promotion and replica recovery, dual HAProxy writable-primary endpoints on 10.104.0.2:16379 and 10.104.0.4:16379, verified TLS health checks using the least-privilege haproxy-health identity, exact CRLF AUTH/INFO replication framing, `min-recv 64` master-role rejection, master-only routing, and successful automatic HAProxy convergence after Sentinel failover. Current primary is ulc-02 (10.104.0.4). Keep the Sentinel-elected primary in place; do not manually fail back.

Acceptance criteria for the completed Valkey HA layer:

```
Three Valkey nodes installed
One primary
Two replicas
Three Sentinel members
Automatic promotion tested
HAProxy follows writable primary
```

Deployment documentation rule

All executed commands, configuration decisions, validation output, failures, and corrections must be recorded in this MDS before repository commit.

The operator performs the final Git commit and push after review.

Next: [09-chirpstack-cloud-cluster.md](#).

Current gateway-facing MQTT authentication correction - 2026-08-28

The target design earlier in this manual shows the gateway-facing Mosquitto backend :8884 with `require_certificate true` and `use_identity_as_username true`. Later live inventory is authoritative for the current runtime: both brokers presently expose :8884 with `require_certificate=false`, `allow_anonymous=false`, no `use_identity_as_username`, and no gateway-specific ACL/plugin. Treat the earlier mTLS block as the **target configuration**, not proof that gateway client-certificate authentication is already commissioned.

The cloud gateway mTLS boundary is therefore still open even though the broker TLS/failover foundation is healthy. Before public `mqtt.<DOMAIN>:8883` is activated, commission the gateway authentication boundary canary-first on ulc-01 then ulc-02: issue one cloud MQTT `clientAuth` certificate whose CN is the authoritative Gateway EUI `0016c001f139a1cb`, add an exact-EUI ACL for the `as923/gateway/0016c001f139a1cb/... event/state/command` hierarchy, enable client-certificate authentication on the gateway-facing :8884 listener, and prove positive mTLS plus cross-EUI/no-certificate denial through the existing anchor :8883 TLS-passthrough path. This work does not require the final public FQDN or Reserved IPv4 and should be completed before provider public activation.

Gateway MQTT cloud client certificate issuance PASS - 2026-08-28

The real physical Gateway EUI 0016c001f139a1cb now has a dedicated cloud MQTT client identity issued on ulc-03 from the commissioned internal CA. Issuance directory: /root/lorawan-pg-ca/gateway-0016c001f139a1cb-issuance-20260828T015149Z; protected transfer bundle is its transfer/ subdirectory. The certificate subject is CN = 0016c001f139a1cb, issuer CN = LoRaWAN PostgreSQL Internal CA, serial D8732205912F3C3AC56E0A01E1E10583, validity 2026-08-28 01:51:51Z through 2027-09-29 01:51:51Z, SHA-256 fingerprint 82:C6:9A:D7:12:5D:8C:45:F3:8F:BA:AB:F9:6E:7E:3B:41:F4:BE:78:95:FE:16:05:FD:58:2E:09:9D:F3:0A:ED, and certificate SHA-256 f348cef6e280dff82722ff908cc96e694faf183eec1dffc55a7abc690cb472d8.

The certificate chains successfully for sslclient, is rejected for sslserver, and its public key matches the RSA-3072 private key. The issuing CA SHA-256 remains 6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb; the existing CA serial-file SHA-256 remained byte-identical at 50df8c462ef9465ab9198284fa1234f0cbfa4f33eb9779ce6d50dd23a618463d because issuance used an explicit random serial. GATEWAY_MQTT_CLIENT_CERT_ISSUANCE=PASS. No Mosquitto, HAProxy, DNS, firewall, or gateway runtime was changed by issuance.

Next boundary: canary-harden ulc-01 gateway-facing Mosquitto :8884 only to require client certificates, map certificate CN to MQTT username, and enforce an exact-EUI as923 gateway ACL. Preserve ChirpStack :8885 and Node-RED :8886. After ulc-01 passes direct mTLS verification with the new gateway certificate, repeat the same bounded rollout on ulc-02.

ulc-01 gateway mTLS canary PASS - 2026-08-28

ulc-01:8884 was hardened from server-TLS-only to gateway client-certificate authentication without changing the ChirpStack :8885 listener. The active gateway listener now has require_certificate true, use_identity_as_username true, allow_anonymous false, and /etc/mosquitto/gateway.acl. The exact Gateway EUI 0016c001f139a1cb is permitted to write only its own as923/.../event/# and state/# topics and read only its own command/# hierarchy. Mosquitto restarted successfully, :8884 and :8885 remained listening, the issued gateway certificate completed TLS and MQTT CONNECT/SUBSCRIBE successfully, and a no-client-certificate connection was rejected. Rollback copy: /etc/mosquitto/gateway-mtls-20260828T015613Z. Next boundary: apply the same bounded change to ulc-02 and prove the same certificate/ACL behavior there before transferring the cloud certificate bundle to Gateway OS.

ulc-02 gateway mTLS rollout PASS - 2026-08-28

ulc-02:8884 now matches the proven ulc-01 gateway-authentication boundary: require_certificate true, use_identity_as_username true, allow_anonymous false, and /etc/mosquitto/gateway.acl. Gateway EUI 0016c001f139a1cb is limited to its own AS923 event/state/command hierarchy. Mosquitto restarted successfully; :8884 and ChirpStack :8885 remained listening. The issued gateway certificate completed TLS, MQTT CONNECT, and own-command SUBSCRIBE successfully, while a no-client-certificate connection was rejected. Rollback copy: /etc/mosquitto/gateway-mtls-20260828T015801Z. Therefore both cloud MQTT broker backends now enforce the intended per-gateway mTLS identity. Next boundary: transfer only the three-file cloud certificate bundle (ca.crt, 0016c001f139a1cb.crt, 0016c001f139a1cb.key) to Gateway OS; do not change the working local Mosquitto topology yet.

ulc-02 gateway mTLS state observed - 2026-08-28

Live read-only inspection after the first rollout wrapper showed ulc-02 is already in the intended gateway mTLS state: /etc/mosquitto/conf.d/tls.conf has listener 8884, require_certificate true, allow_anonymous false, use_identity_as_username true, and acl_file /etc/mosquitto/gateway.acl; /etc/mosquitto/gateway.acl exists as 0640 root:mosquitto; per_listener_settings true remains active; and the dedicated ChirpStack listener remains 10.104.0.4:8885 with its own password/ACL files. Therefore do not reapply the mutation. The remaining boundary is a direct positive mTLS/MQTT proof with gateway certificate CN 0016c001f139a1cb plus a no-client-certificate rejection proof.

Both cloud gateway MQTT brokers mTLS acceptance PASS - 2026-08-28

Direct verification from ulc-03 against ulc-02:8884 using the issued gateway identity CN = 0016c001f139a1cb passed TLS, MQTT CONNECT, and subscription to the gateway's own as923/gateway/0016c001f139a1cb/command/# hierarchy. A client without a certificate was rejected. Combined with the earlier ulc-01 canary proof, both gateway-facing Mosquitto backends now enforce per-gateway mTLS and the exact EUI ACL while preserving the dedicated ChirpStack :8885 listeners. The cloud broker-side gateway authentication boundary is therefore complete. Next: transfer only ca.crt, 0016c001f139a1cb.crt, and 0016c001f139a1cb.key from the protected ulc-03 issuance bundle to Gateway OS; do not alter the local loopback broker topology during certificate import.

Supporting runbook: 14-observability-alerting-and-logging.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

14. Minimal POC Observability and Evidence Capture

Status: 14S SERVER-ONLY HARNESS PRE-STAGING MAY PROCEED / FINAL PHASE 14 AFTER 13B. The final healthy-baseline evidence run still occurs only after Phase 13B against the fully commissioned system. While hardware/provider/Fabric dependencies are unavailable, an interim **14S server-only harness** may be run against the already commissioned server components to prove commands, permissions, UTC timestamps, redaction, and evidence-directory structure early. 14S is preparation evidence, not the final Phase 14 PASS.

The POC needs **repeatable evidence**, not a production monitoring platform.

Do not deploy Prometheus, Alertmanager, Loki, or another monitoring stack merely to prove the three-server architecture. For a few sensors, command-line checks, service logs, a small Grafana telemetry dashboard, and timestamped evidence folders are enough.

The important rule is:

First prove the evidence harness on a fully healthy baseline. Only after Phase 14B passes should the same capture set be used before, during, and after Phase 15 failures.

Phase 14S server-only evidence-harness pre-stage

Before the final Phase 13B exists, the server-only portion of the harness may be exercised once with a SERVER-PRESTAGE-... run ID. Populate only a before/ state; do not create a fake failure state. Prove non-secret capture for the components that are actually deployed: host resources, etcd, Patroni/PostgreSQL/TimescaleDB, PgBouncer/HAProxy, Valkey/Sentinel, Mosquitto, ChirpStack, Node-RED A/B state, Grafana, OpenBao, and telemetry.fabric_outbox. Record hardware/provider/Fabric-adapter/gateway-integrity fields as DEFERRED or BLOCKED rather than inventing values.

The 14S pass marker is SERVER_ONLY_EVIDENCE_HARNESS=PASS. This means the server-side commands/permissions are ready. It does **not** authorize Phase 15 and does not replace the final healthy-baseline run.

Phase 14S current execution state - 2026-08-29

The first server-only healthy-state capture created run ID SERVER-PRESTAGE-20260829T133354Z under /home/opsadmin/lorawan-ha-evidence/SERVER-PRESTAGE-20260829T133354Z. It is read-only: no service restart, configuration mutation, backup, or failure injection is part of this run.

The following gates are authoritative PASS and must not be repeated merely because a later harness step stopped:

```
THREE_HOST_EVIDENCE=PASS
ETCD_3_MEMBER_HEALTH=PASS
PATRONI_1_LEADER_2_REPLICAS=PASS
POSTGRES_TIMESCALE_OUTBOX=PASS
NODE_RED_SINGLE_ACTIVE=PASS
```

At capture time Patroni reported 10.104.0.2 as the single leader and 10.104.0.4 / 10.104.0.8 as replicas. Node-RED A was running|0|healthy with HTTP 200 on 127.0.0.1:1880; Node-RED B remained fenced/stopped. The recurring PostgreSQL locale warning remained non-blocking.

The first Grafana step printed GRAFANA_CONTAINER=FAIL, but this was a discovery-harness false negative rather than a service failure. The harness searched Docker's displayed image text for the word grafana; because the immutable image was displayed as digest prefix 3fd54ae12146, no candidate was found. A direct read-only diagnostic then proved the Compose-labeled container grafana was running, restart count 0, OOMKilled=false, memory limit 536870912, listener 127.0.0.1:3000, and /api/health returned database ok, Grafana 13.2.0, commit f681b1359f6a0b8ecb9f2c49a88ac72b75bde73b. GRAFANA_RUNTIME_DISCOVERY=PASS is authoritative.

The corrected resume preserved Steps 1-6, then passed GRAFANA_HEALTH=PASS using the Compose service label / loopback port rather than image-name text, and passed CHIRPSTACK_TWO_NODE_RUNTIME=PASS on ulc-01 and ulc-02. These are now authoritative and must not be rerun merely because the next OpenBao probe stopped.

The OpenBao step then failed before collecting any member state because docker exec openbao bao status - format=json used the CLI default https://127.0.0.1:8200. The commissioned OpenBao listeners are intentionally bound to the host-private addresses 10.104.0.2:8200, 10.104.0.4:8200, and 10.104.0.8:8200; container loopback is not a commissioned listener. The resulting dial tcp 127.0.0.1:8200: connect: connection refused is therefore a harness-address defect, not evidence of OpenBao service failure. Resume only from OpenBao onward and set BAO_ADDR=https://<that-node-private-ip>:8200 plus BAO_CACERT=/openbao/tls/ca.crt for every in-container bao status call.

The final resume set BAO_ADDR explicitly to each commissioned private OpenBao API address and BAO_CACERT=/openbao/tls/ca.crt. All three members returned initialized=true, sealed=false, and ha_enabled=true, so OPENBAO_3_NODE_HEALTH=PASS is authoritative. The harness then wrote the provider/hardware/Fabric/gateway-integrity items as DEFERRED or BLOCKED, passed the evidence secret-sanity scan, normalized the evidence tree to mode 0700 directories / 0600 files, and generated a final SHA-256 manifest covering the complete 16-file evidence set. DEFERRED_BLOCKED_RECORD=PASS, EVIDENCE_SECRET_GATE=PASS, EVIDENCE_SHA256=PASS, and EVIDENCE_FILESYSTEM_PROTECTION=PASS are authoritative.

SERVER_ONLY_EVIDENCE_HARNESS=PASS is therefore complete for run SERVER-PRESTAGE-20260829T133354Z. The evidence directory is /home/opsadmin/lorawan-ha-evidence/SERVER-PRESTAGE-20260829T133354Z. No service restart, configuration mutation, backup, or failure injection occurred during this 14S run. Do not repeat 14S merely to continue setup or begin a new chat; the final Phase 14 healthy-baseline evidence capture still occurs after final Phase 13B against the fully commissioned system.

Phase 14 healthy-baseline dry run

After the final Phase 13B snapshot exists, create a normal evidence directory and populate the before/ state for every deployed component. Do not create a fake during/ failure state. Verify all commands run non-interactively with the intended least-privilege access, timestamps are UTC, and no secrets are copied into evidence.

The final dry run must include Node-RED, Grafana, OpenBao, both Fabric adapters, the latest normal Fabric commit, the current Reserved-IP owner, and the final Phase 13B backup-set identifier.

14.1 What every Phase 15 failure run must answer

For each run be able to answer:

```
what failed?
when was the fault injected?
which PostgreSQL / Valkey / OpenBao roles changed?
did etcd / Sentinel / OpenBao quorum survive?
which Droplet owned the Reserved IPv4 before/after failure?
did the public-ingress failover agent move it, and how long did that take?
which MQTT broker served after failure?
which ChirpStack instance remained usable?
did a NEW post-fault real uplink succeed?
did Timescale telemetry remain consistent?
did the Fabric outbox remain durable?
was Fabric adapter execution deployed or BLOCKED?
did any 2-GiB host OOM or become swap-bound?
what was the measured recovery time?
was full 3-node redundancy restored before the next run?
```

14.2 Create one evidence directory per test

On the administration workstation or another approved evidence host:

```
RUN_ID="HA-$(date -u +%Y%m%dT%H%M%SZ)-<TEST>-R1"
EVIDENCE="$HOME/lorawan-ha-evidence/$RUN_ID"
umask 077
mkdir -p "$EVIDENCE"/{before,during,after,logs}
printf '%s\n' "$RUN_ID" | tee "$EVIDENCE/RUN_ID.txt"
date -u --iso-8601=seconds | tee "$EVIDENCE/created-at.txt"
```

Create notes.md containing only non-secret facts:

```
run ID:
failure being tested:
fault method:
staging Gateway EUI:
staging DevEUI:
pre-fault test_sequence/event key:
Reserved IPv4 owner before fault:
Reserved IPv4 owner after recovery:
Reserved-IP reassignment action/time:
fault timestamp UTC:
first successful post-fault event UTC:
RTO:
unexpected behavior:
restore timestamp UTC:
final result: PASS / FAIL / BLOCKED
```

Do not copy passwords, private keys, AppKeys, bearer tokens, complete DSNs, or OpenBao recovery material into the evidence folder.

14.3 Host resource snapshot

Run on **ha-01**, **ha-02**, and **ha-03** before/during/after the test and save the output under the matching evidence subdirectory:

```
hostname
printf '\n=== UTC ===\n'
date -u --iso-8601=seconds
printf '\n=== MEMORY ===\n'
free -h
printf '\n=== CPU / LOAD ===\n'
uptime
command -v vmstat >/dev/null && vmstat 1 5 || true
printf '\n=== DISK ===\n'
```

```
df -h /
printf '\n=== SERVICES ===\n'
systemctl --failed --no-pager
printf '\n=== CONTAINERS ===\n'
docker stats --no-stream 2>/dev/null || true
printf '\n=== OOM ===\n'
journalctl -k --since today --no-pager | grep -Ei 'oom|out of memory|killed process' || true
```

Why: with 2-GiB shared-CPU nodes, an OOM **or a functional failover failure caused by sustained CPU contention** is a sizing result. Do not restart the process and erase the evidence before recording it. Use the functional recovery criteria rather than inventing a universal CPU-utilization threshold.

14.4 etcd snapshot

From an approved administrative shell, using the currently tested east-west endpoints:

```
ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  endpoint health

ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  endpoint status --write-out=table

ETCDCTL_API=3 etcdctl \
  --endpoints=http://10.104.0.2:2379,http://10.104.0.4:2379,http://10.104.0.8:2379 \
  member list --write-out=table
```

These commands match the current HTTP-only etcd deployment on 10.104.0.0/20. Add CA/client options only after an etcd TLS rollout has been completed and tested.

Record:

```
healthy member count
leader/member IDs
revision
3/3 before
2/3 during one-host/member loss
3/3 after restore
```

A 2/3 state is acceptable only **during the single-failure test**. Do not begin another fault until it is back to 3/3.

14.5 PostgreSQL / Patroni snapshot

Use the exact pinned Patroni configuration:

```
patronictl -c <PATRONI_CONFIG> list <PG_SCOPE> --extended
```

Then verify the normal client route from the host being tested:

```
psql \
  'host=pgbouncer.internal.lorawan.com port=6432 dbname=postgres user=<MONITOR_ROLE> sslmode=verify-full' \
  -c 'SELECT now() AT TIME ZONE '\''UTC'\'' AS utc_time, inet_server_addr(), pg_is_in_recovery();'
```

Expected for the routed server:

```
pg_is_in_recovery = false
```

Record:

```
primary before/after
replica members
lag where available
promotion/switchover time
server address returned through PgBouncer/HAProxy
```

Both logical databases must remain present:

```
chirpstack
lorawan_telemetry
```

14.6 TimescaleDB snapshot

Against lorawan_telemetry through the normal PgBouncer path:

```
psql \
'host=pgbouncer.internal.lorawan.com port=6432 dbname=lorawan_telemetry user=telemetry_reader
sslmode=verify-full' \
-c "SELECT extname, extversion FROM pg_extension WHERE extname='timescaledb';"

psql \
'host=pgbouncer.internal.lorawan.com port=6432 dbname=lorawan_telemetry user=telemetry_reader
sslmode=verify-full' \
-c "SELECT hypertable_schema, hypertable_name FROM timescaledb_information.hypertables ORDER BY 1,2;"
```

Required hypertables:

```
telemetry.uplinks
telemetry.measurements
```

Also record the latest real measurement without assuming a sensor-specific SQL column:

```
SELECT time, dev_eui, metric_name, metric_value, metric_text, metric_bool, unit, quality
FROM telemetry.measurements
ORDER BY time DESC
LIMIT 20;
```

After PostgreSQL promotion, repeat the extension/hypertable queries. A promoted member that cannot load TimescaleDB is not an acceptable HA recovery.

14.7 PgBouncer / HAProxy snapshot

On each host that owns PgBouncer:

```
SHOW POOLS;
SHOW SERVERS;
SHOW STATS;
```

Use the protected PgBouncer admin/stats DSN and redact credentials from saved output.

Record:

```
client waiting count
server connection count
```

```
server address before/after promotion
unexpected auth failures
reconnect behavior
```

For the PostgreSQL HAProxy route, a read-only SQL query through :15432 must reach a writable primary. For MQTT/Valkey/OpenBao, use the component-specific checks below rather than assuming an HAProxy process means every backend is healthy.

14.8 Valkey / Sentinel snapshot

Check all three data nodes:

```
valkey-cli --tls --cacert <VALKEY_CA> \
-h <VALKEY_PRIVATE_IP> -p 6379 \
-a '<LOAD_FROM_PROTECTED_SOURCE>' ROLE
```

Check all three Sentinels:

```
valkey-cli --tls --cacert <VALKEY_CA> \
-h <SENTINEL_PRIVATE_IP> -p 26379 \
-a '<LOAD_SENTINEL_SECRET_PROTECTED>' \
SENTINEL CKQUORUM lorawan-valkey

valkey-cli --tls --cacert <VALKEY_CA> \
-h <SENTINEL_PRIVATE_IP> -p 26379 \
-a '<LOAD_SENTINEL_SECRET_PROTECTED>' \
SENTINEL GET-MASTER-ADDR-BY-NAME lorawan-valkey
```

Finally query the **application endpoint**:

```
valkey-cli --tls --cacert <VALKEY_CA> \
--sni valkey.internal.lorawan.com \
-h <LOCAL_HAPROXY_PRIVATE_IP> -p 16379 \
-a '<LOAD_FROM_PROTECTED_SOURCE>' ROLE
```

Record:

```
primary before/after
replica count
Sentinel CKQUORUM result
promotion time
HAProxy endpoint role
```

The application endpoint must always report the writable primary/master.

14.9 MQTT snapshot

The cloud port ownership must be:

```
HAProxy public gateway service:      :8883 on ulc-01/ulc-02 anchors
HAProxy ChirpStack workload service:  :18883 on ulc-01/ulc-02 private VPC
HAProxy Node-RED ingest service:      :18884 on ulc-03 private VPC
Mosquitto gateway-facing TLS:         :8884 on ulc-01/ulc-02
Mosquitto Node-RED mTLS:              :8886 on ulc-01/ulc-02
Mosquitto ChirpStack workload TLS:    :8885 on ulc-01/ulc-02
Gateway-local buffer:                 127.0.0.1:1883 on Raspberry Pi only
```

On cloud hosts:

```
sudo ss -lntp | grep -E ':(8883|8884|8885|18883|18884)\b' || true
```

Record from HAProxy/Mosquitto logs:

```
preferred broker health
backup broker health
gateway disconnect/reconnect timestamp
Node-RED/ChirpStack MQTT reconnect if affected
first successful post-switch uplink
```

On the Raspberry Pi also record the local queue/persistence indicators described by the gateway buffer manual before, during, and after a backhaul/broker test.

Do not call successful reconnection proof of broker-session replication. The POC is testing service failover plus the gateway's bounded persistent uplink buffer.

14.10 ChirpStack snapshot

Record:

```
ChirpStack-1 process/HTTP health
ChirpStack-2 process/HTTP health
public https://chirpstack.<DOMAIN> result
gateway last-seen
OTAA result when that test is selected
fresh uplink event ID / sequence
downlink result when that test is selected
```

A running process is not enough. The recovery timer ends only on the first **new post-fault** staging-device uplink accepted through the surviving path, using the definition in [15-failover-chaos-and-acceptance-testing.md](#).

14.10A Node-RED and Grafana snapshot

On ulc-03, capture:

```
Node-RED container state/restart count
Node-RED listener = 127.0.0.1:1880 only
Node-RED private MQTT route :18884 listener/backend health
latest real telemetry event_key/test_sequence stored through Node-RED
Node-RED exported flow hash/reference
Grafana container state/restart count
Grafana listener = 127.0.0.1:3000 only
Grafana datasource identity = telemetry_reader
latest dashboard-visible reading time/age
Grafana dashboard export hash/reference
```

Use a read-only database query to correlate the latest telemetry row with the dashboard. Do not use the Grafana admin credential as evidence that the database path works.

14.11 OpenBao snapshot

From an approved OpenBao admin environment:

```
bao status
bao operator raft list-peers
```

Through the stable HAProxy endpoint:

```
curl --fail --silent --show-error \
  --cacert /etc/lorawan-pki/openbao/ca.crt \
  'https://openbao-kms.internal.<DOMAIN>:18200/v1/sys/health?standbyok=true'
```

Record:

```
member count
active node
sealed/unsealed state
healthy Raft peer set = 3/3
Transit sign/verify result through stable :18200
current Transit key-version metadata without private key material
Phase 15 will record 2/3 behavior during one-member loss
```

Do not put root tokens, recovery shares, or unseal material in the evidence folder.

14.12 Fabric outbox and conditional adapter evidence

The outbox is a durable application boundary and must be monitored independently from adapter process health.

Record status counts with a read-only database role using the deployed schema:

```
SELECT status, count(*)
FROM telemetry.fabric_outbox
GROUP BY status
ORDER BY status;
```

Also record oldest eligible/pending age using the actual schema timestamp column documented by the outbox migration.

Current commissioned adapter evidence

Record:

```
ULC-01 adapter-1 enabled state + process health + worker_id
ULC-02 adapter-2 write-disabled/fenced state
outbox eligibility: finalized_payload present; v2 verifier status=verified
pending / processing / failed / confirmed counts
reconciling count + oldest age
needs_attention count + oldest age
legacy submitted_unknown count only as migration compatibility
live/expired lease evidence
OpenBao HA/sign/verify health
HRC Gateway reachability and TLS identity
Fabric tx ID + HRC record ID + commit/reconcile result for sampled work
```

Do not infer that pending with attempts=0 is a Fabric outage. First prove the row satisfies the current claim predicate. Do not report ULC-02 as an active writer merely because its container/image exists.

During an external Fabric outage the expected pattern is:

```
telemetry still inserts
outbox grows/waits
```

14.12A Gateway-integrity evidence-service capture

Add this section only after the reviewed v2 evidence services are actually deployed. It is distinct from the generic test evidence folder: these files prove the **security lineage services themselves**.

Capture at minimum:

```
gateway-journal.txt
  journal version/executable hash
  process/restarts
  last sequence/record hash
  open/closed segment IDs
  storage usage/reserve
  unuploaded segment count/bytes
  uploader retry state
  latest accepted checkpoint receipt age

evidence-storage.txt
  backend/path identity without secrets
  free space/capacity
  object count/oldest backlog where meaningful
  no-overwrite/append-only configuration evidence

evidence-ingest.txt
  ingest-1 + ingest-2 process/version/image digest
  per-replica health/restart/OOM/resource state
  listener/SNI/backend identity
  accepted/rejected/conflicting upload counters
  latest accepted checkpoint/segment receipt
  duplicate retry convergence to one identity
  unknown/unauthenticated-client rejection evidence

mqtt-evidence-collector.txt
  collector-1 + collector-2 process/version/image digest
  four persistent broker sessions: each collector -> broker-1 + broker-2
  dedicated client IDs/identity names without secrets
  subscription scope
  collector lag/latest captured gateway event
  deterministic capture_key convergence
  publish/command permission denial result

evidence-verifier.txt
  verifier-1 + verifier-2 process/version/image digest
  identical trusted-decoder digest
  current worker_id/lease state
  oldest pending work
  pending/verified/evidence_gap/integrity_failure counts
  unmatched journal/MQTT/application counts
  checkpoint conflict count
  trusted-decoder mismatch count
  expired-lease reclaim evidence from commissioning/Phase15 as appropriate

trusted-decoder.txt
  decoder_id/version or code hash
  fixed-vector self-test result
  latest raw_app_data_sha256/normalized_digest_sha256 references for the selected event

gateway-evidence-state.txt
  selected source event key/observed_at
  verification_id/status/reason
  journal segment/sequence/hash references
  checkpoint ID
  MQTT gateway_event_id
  verified_at/update age

checkpoint-freshness.txt
```

```
latest accepted checkpoint per gateway
sequence/segment and server_received_at
age relative to current UTC
```

Do not copy raw private keys, bearer tokens, unrestricted payload dumps, or OpenBao/Fabric credentials into the Phase 14 folder. Raw segments/events remain in the protected evidence store; Phase 14 records stable IDs, hashes, status, and bounded sanitized excerpts.

Diagnostic rule:

```
telemetry fresh + MQTT fresh + checkpoint stale
```

means delivery is healthy while the evidence upload/ingest path is degraded. Keep those states separate.

14.13 Grafana

Grafana is visual confirmation, not the source of truth for failover timing.

Use the small dashboard for:

```
latest sensor value
latest timestamp / reading age
RSSI/SNR where stored
small history view
```

Keep a slow refresh interval appropriate to the few-sensor POC. Do not change the 2-GiB resource profile by running aggressive dashboard polling during the sizing test.

14.14 Before / during / after capture pattern

For each component, save the same command output three times:

```
<run-id>/
before/
  hosts-*.txt
  public-ingress.txt
  etcd.txt
  patroni.txt
  timescale.txt
  pgbouncer-*.txt
  valkey-sentinel.txt
  mqtt.txt
  chirpstack.txt
  openbao.txt
  outbox.txt
  gateway-journal.txt           # when v2 deployed
  evidence-storage.txt         # when v2 deployed
  evidence-ingest.txt          # when v2 deployed
  mqtt-evidence-collector.txt  # when v2 deployed
  evidence-verifier.txt        # when v2 deployed
  trusted-decoder.txt          # when v2 deployed
  gateway-evidence-state.txt    # when v2 deployed
  checkpoint-freshness.txt     # when v2 deployed
during/
  same files
after/
  same files
logs/
  only the bounded/sanitized excerpts needed to explain the transition
notes.md
```

Screenshots may supplement the folder, but do not replace machine-readable command output.

14.15 Pass condition

Observability setup is ready only when a **healthy-state dry run** produces, without undocumented memory:

```
UTC/run identifier and Phase 13B backup ID
host CPU/memory/swap/OOM evidence
etcd 3/3
Patroni primary/replica/lag state
PgBouncer pool/wait state
Timescale extension/hypertable/latest telemetry state
Valkey/Sentinel primary/quorum state
MQTT backend + HAProxy listener state
ChirpStack-1/2 health + gateway last-seen
Reserved IPv4 current owner + failover timer state
Node-RED state + current flow hash + latest stored event
Grafana state + dashboard hash + latest visible reading age
OpenBao 3/3 + Transit normal-path result
when v2 is selected: journal/uploader + ingest + evidence storage + MQTT collector + verifier + trusted
decoder + checkpoint freshness + verification-state evidence
when v2 is selected: one real staging lineage already reached verified before any Fabric v2 seal
Fabric adapter-1/2 + outbox counts + last confirmed tx
```

The harness must also have commands/templates ready to populate the later during/ and after/ folders, but **no fault is injected in Phase 14**.

Production monitoring, centralized logs, long retention, alert routing, and SLO dashboards are later hardening, not prerequisites for this small architecture.

Next: [14b-pre-test-commissioning-gate.md](#). Phase 15 remains blocked until that gate passes.

Supporting runbook: 05-troubleshooting.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Operations 5. Troubleshooting the Buffered Gateway

Troubleshoot the lowest failing layer first. Do not reflash the gateway or change the radio region as a first response. Unless a section says otherwise, run shell commands over SSH on the Raspberry Pi gateway.

Use one known device uplink while testing. A healthy result at one layer proves only that layer; for example, an MQTT connection does not prove the RAK5146 is receiving the correct RF channels.

Last seen: Never — checkpoint flow

When the gateway is registered but ChirpStack shows **Last seen: Never**, use these checkpoints in order and stop at the first failure:

1. **Radio / Concentratord** — active SX1302/SX1303 process is running, RAK5146 initializes on `/dev/spidev0.0`, and `logread -e chirpstack-concentratord | tail -n 100` reports the real Gateway ID retrieved EUI. Ignore inactive SX1301 gateway IDs.
2. **Gateway registration** — that exact 16-hex EUI is manually registered in ChirpStack. Missing from the list means not registered; present with Last seen Never means registered but no gateway event has been processed yet.
3. **MQTT Forwarder** — backend is Concentratord, server is `tcp://127.0.0.1:1883`, topic prefix matches the region, JSON is disabled, and TLS fields are empty.
4. **Local Mosquitto** — listener is only `127.0.0.1:1883`, Forwarder connects, and persistent state can be saved to `mosquitto.db`.
5. **TLS** — `openssl s_client` verifies CA, hostname, server certificate, and the gateway client certificate/key with return code 0.
6. **MQTT authentication / ACL** — a certificate-authenticated `mosquitto_pub` receives `CONNACK (0)` and `PUBACK (... RC:0)` on the gateway's own allowed topic.
7. **Two bridges** — `gw-up-<GATEWAY_EUI>` and `gw-down-<GATEWAY_EUI>` remain connected; normal keepalive `PINGRESP` messages are present.
8. **Server broker** — direct `mosquitto_sub` against the broker sees `<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/stats`. Binary Protobuf output is normal.
9. **ChirpStack configuration** — ChirpStack is running without TOML errors; `enabled_regions` contains the region; the active v4 region file uses `[[regions]]`, enables the MQTT backend, and has matching topic prefix/broker credentials. Keep backup TOMLs outside the mounted ChirpStack config directory.
10. **ChirpStack UI** — Last seen becomes recent. Only now move to real RF-device join/uplink testing.

1. Gateway does not boot or cannot be reached

What appears to be failing: the Raspberry Pi, Gateway OS, Ethernet/Wi-Fi path, or management addressing.

Most likely causes: inadequate power, a failed or corrupted SD card, no network link, a changed DHCP address, or the wrong image for the Raspberry Pi 4B.

From the commissioning workstation, check the router or DHCP server for the gateway lease, then test the observed address:

```
ping <GATEWAY_IP_ADDRESS>
ssh root@<GATEWAY_IP_ADDRESS>
```

A healthy gateway has a stable lease, responds on the management network, and opens LuCI or SSH. No lease usually points to power, boot media, or link state. A lease with no SSH response points to addressing, firewall, or a partially booted system.

The smallest safe fix is to verify the power supply and cable, reseal the SD card with power removed, and test the approved Gateway OS Base image on a spare card. Preserve the original card for log or configuration recovery. Verify the fix by rebooting once and confirming that the same management address is reachable again.

2. Concentratord has no stable Gateway ID

What appears to be failing: the service that owns the RAK5146 concentrator.

Run on the gateway:

```
monit status
uci show chirpstack-concentratord
logread -e chirpstack-concentratord
```

A healthy result shows the Concentratord service running, the SX1302/SX1303 chipset and RAK5146 shield selected, the confirmed channel plan, and one stable 16-hexadecimal Gateway ID.

Interpret abnormal output before changing anything:

```
wrong shield or chipset -> RAK5146 profile mismatch
SPI or reset error      -> HAT seating, power, reset mapping, or hardware fault
calibration loop        -> frequency variant, clock, profile, or hardware problem
Gateway ID changes      -> unstable concentrator identity or hardware state
```

For a profile mismatch, correct only the observed chipset, shield, or channel-plan value in **ChirpStack > Concentratord**. For SPI, reset, or hardware errors, power down before reseating the HAT. Do not repeatedly change the Gateway EUI in ChirpStack. Verify the fix by rebooting and confirming the same Gateway ID appears without restart or calibration loops.

3. MQTT Forwarder cannot publish to the local broker

What appears to be failing: the gateway-local hop from MQTT Forwarder to Mosquitto.

Run on the gateway:

```
uci show chirpstack-mqtt-forwarder
logread -e chirpstack-mqtt-forwarder
```

```
logread -e mosquitto
ss -lntp 2>/dev/null | grep ':1883' || netstat -lntp | grep ':1883'
```

Healthy configuration uses `tcp://127.0.0.1:1883`, Protobuf, backend concentrator, QoS 1, and a Mosquitto listener bound only to `127.0.0.1:1883`.

Abnormal results mean:

```
connection refused    -> Mosquitto is stopped, invalid, or listening on another port
WAN broker configured -> MQTT Forwarder bypasses the persistent local buffer
QoS is not 1          -> the local broker does not provide the intended durable acknowledgement
0.0.0.0 or LAN listener-> the local broker is exposed beyond the gateway
```

Validate the Mosquitto configuration before restarting it, then correct only the local address, listener, or QoS setting that differs. Verify with one real uplink: MQTT Forwarder logs should stop reporting connection errors and Mosquitto should receive the regional gateway event topic.

4. Local broker receives messages but the remote bridge is offline

What appears to be failing: DNS, route/4G, system time, TLS identity, broker ACLs, or the remote MQTT service.

Run on the gateway:

```
logread -e mosquitto
nslookup <MQTT_BROKER_FQDN>
ip route
date -u
df -h /etc/mosquitto/data
ls -l /etc/mosquitto/data/mosquitto.db
```

Healthy output shows the broker name resolving to the intended endpoint, a usable default route, correct UTC time, sufficient free space, a persistent `mosquitto.db`, and successful bridge authentication for both `gw-up-<GATEWAY_EUI>` and `gw-down-<GATEWAY_EUI>`.

Use the first matching failure:

```
DNS or route failure -> WAN path unavailable; repair DNS, APN, or route and leave the queue running
unknown CA or hostname -> wrong trust file or certificate SAN; install the intended CA and hostname
client rejected -> expired/wrong certificate, EKU, CN, or private-key mismatch
not authorized -> certificate identity, Gateway EUI, region prefix, or ACL mismatch
queue grows -> remote service is unavailable or draining more slowly than new traffic arrives
queue limit reached -> the outage exceeded finite capacity and some delivery may be lost
```

Never disable certificate verification to make the bridge connect. Correct the endpoint, trust, certificate/key pair, or exact topic ACL.

Isolate the connection in three layers:

```
# A. TCP only
nc -w 3 <MQTT_BROKER_FQDN> 8883 </dev/null
echo $?

# B. TLS, hostname, CA, and client-certificate verification
openssl s_client \
  -connect <MQTT_BROKER_FQDN>:8883 \
  -servername <MQTT_BROKER_FQDN> \
  -verify_hostname <MQTT_BROKER_FQDN> \
  -verify_return_error \
  -CAfile /etc/mosquitto/certs/ca.crt \
  -cert /etc/mosquitto/certs/<GATEWAY_EUI>.crt \
```

```

-key /etc/mosquitto/certs/<GATEWAY_EUI>.key \
</dev/null

# C. MQTT authentication + write ACL
mosquitto_pub \
-h <MQTT_BROKER_FQDN> -p 8883 \
--cafile /etc/mosquitto/certs/ca.crt \
--cert /etc/mosquitto/certs/<GATEWAY_EUI>.crt \
--key /etc/mosquitto/certs/<GATEWAY_EUI>.key \
-i gw-cert-test-<GATEWAY_EUI> \
-t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/state/test' \
-m 'mtls-auth-test' -q 1 -d

```

Interpret them separately: TCP exit 0 proves only route/port; TLS must report verification return code 0; MQTT CONNACK (0) proves authentication and PUBACK (... RC:0) proves the ACL allowed publication. Verify the final bridge fix by observing reconnect, queue drain without deleting mosquitto.db, and a new gateway event at the remote broker.

5. Remote broker receives gateway events but ChirpStack does not

What appears to be failing: the server-side MQTT-to-ChirpStack path.

Run this isolation test on the application server before relying on container stdout logs:

```

sudo timeout 45 docker exec mosquitto mosquitto_sub \
-h 127.0.0.1 -p 1883 \
-u chirpstack -P <CHIRPSTACK_MQTT_PASSWORD> \
-t '<REGION_TOPIC_PREFIX>/gateway/<GATEWAY_EUI>/event/stats' \
-v -C 1

```

A received Protobuf packet proves the gateway bridge reached the server broker even when `docker logs mosquitto --since 5m` is empty. Broker topic delivery and application stdout logging are different evidence.

Then confirm the active ChirpStack region MQTT backend subscribes to the same case-sensitive region prefix emitted by Gateway OS and uses the intended broker identity. A healthy result shows ChirpStack updating the registered Gateway EUI's last-seen time. If the broker has the topic but ChirpStack remains silent, check ChirpStack process health, TOML parse errors, `enabled_regions`, `v4 [[regions]]`, explicit MQTT backend enablement, topic prefix, credentials, and Protobuf handling.

Make the smallest correction to the ChirpStack region configuration or broker ACL, restart only the affected service, and verify with a fresh uplink. Do not add ChirpStack Gateway Bridge; this architecture sends Concentrator Protobuf topics directly to ChirpStack's MQTT region backend.

6. Duplicate application or telemetry rows

What appears to be failing: downstream idempotency, not gateway transport.

QoS 1 and reconnects can legitimately redeliver the same event. Check that Node-RED uses ChirpStack `deduplicationId` or the documented stable fallback event key and that the TimescaleDB unique indexes exist.

A healthy replay leaves one uplink row and one row per normalized metric for the event key. More than one row means the application or database duplicate guard is missing or uses an unstable key. Fix the event-key or unique-index path and replay the same sanitized event again. Do not reduce gateway QoS to hide duplicate storage.

7. Buffered messages disappear after reboot

What appears to be failing: Mosquitto persistence or the selected storage path.

Run on the gateway before and after a controlled reboot:

```
df -h /etc/mosquitto/data
mount | grep -E ' /tmp | /etc/mosquitto/data'
ls -l /etc/mosquitto/data/mosquitto.db
logread -e mosquitto
```

Healthy behavior keeps the data path on persistent storage, saves a non-empty database with the intended owner, and reopens the queue after restart. A missing database, tmpfs path, ownership error, or repeated persistence warning means the queue is not durable.

Correct the path, permissions, or persistence setting without deleting the only queue database. Verify with the documented WAN-loss, reboot, reconnect, and drain test. Do not claim durable buffering until a real event survives that sequence.

8. A stale downlink transmits after recovery

What appears to be failing: downlink session cleanup or an unintended broker queue.

Confirm the downlink bridge uses `cleansession true` and the command topic direction is `in 0`. Check that no second client session, retained message, or automation path is publishing the old command.

Healthy behavior drops commands whose Class A receive window has passed. Any stale transmission means automation must remain disabled. Remove the unintended persistent session or retained command, reconnect, then test with a harmless device command whose receive window and result can be observed.

9. RF uplinks or downlinks fail while MQTT is connected

What appears to be failing: the radio, device registration, or LoRaWAN scheduling layer.

Compare the antenna band and connection, RAK5146 frequency variant, Concentrator channel plan, ChirpStack region, device profile, device regional variant, OTAA root-key reference, frame counters, class, receive windows, gateway time, FPort, and legal transmit limits.

Healthy commissioning shows a JoinRequest, JoinAccept, a later data uplink, and—when required—a safe downlink in the expected receive window. A connected MQTT bridge with no RF event means backhaul is healthy but radio or device settings are not. Correct the first mismatched region, profile, key reference, or antenna condition and repeat one join/uplink test.

10. UDP Forwarder traffic appears

What appears to be failing: the supported architecture has been bypassed.

Run on the gateway:

```
uci show chirpstack-udp-forwarder
monit status
```

Healthy output has no UDP server configured and the UDP Forwarder disabled. Remove only the unintended UDP server entries, disable the service, and verify that MQTT Forwarder continues delivering a real uplink through the local broker.

11. Integrity journal does not advance

What appears to be failing: the independent Concentratord evidence path, not necessarily MQTT delivery.

Check the reviewed journal service status, its configured read-only Concentratord interface, persistent evidence directory, current sequence, open segment, and journal-specific logs/metrics using the implementation's documented diagnostics.

Healthy behavior shows a new sequence for a known real uplink while MQTT Forwarder independently publishes the corresponding local event.

Interpret the failure by layer:

```
Concentratord healthy, MQTT healthy, journal stopped
-> journal implementation/service/permission fault

journal process running, sequence unchanged
-> source subscription/interface or filter fault

sequence advances, writes fail
-> evidence storage/permissions/free-space fault
```

Do not point the journal at Mosquitto merely to make its counter move. Repair the independent source path and mark the unjournaled interval as an evidence gap.

12. Journal hash or sequence continuity fails

A complete record with an invalid hash, a broken previous-hash link, an unexplained sequence reset, or a segment-hash conflict is not routine log corruption.

Preserve the files and current storage image before attempting repair. Compare against the latest server checkpoint and the server's independently captured gateway MQTT events. Do not renumber records, recalculate a replacement chain in place, or delete the conflicting checkpoint.

Use:

```
missing required object with no contradictory bytes -> evidence_gap
proven hash/payload/rollback conflict               -> integrity_failure
```

13. Cloud checkpoint is stale but MQTT is healthy

What appears to be failing: the evidence uploader/authentication/API path.

Check independently:

```
DNS/route to evidence endpoint
TLS hostname and CA
per-gateway upload identity
latest local closed segment
unuploaded segment count/bytes
```

```
latest accepted server checkpoint age
API rejection/conflict logs
```

Do not treat healthy MQTT as proof that evidence upload works. Restore the uploader and verify the next accepted checkpoint extends the previous server anchor.

14. Journal and remote MQTT payload disagree

Freeze v2 evidence promotion for the affected lineage. Preserve:

```
journal segment/object
remote MQTT captured event/object
Gateway EUI
PHYPayload digests
uplink/context identifiers
timestamps
forwarder/broker versions and logs
```

A payload conflict after the Concentrator split can indicate journal corruption, MQTT Forwarder/delivery-path alteration, wrong correlation, or compromise. Do not let Node-RED choose which copy is authoritative. Use the server verifier's deterministic matching contract.

15. Gateway evidence storage approaches its limit

Restore the evidence upload path before the emergency OS reserve is threatened. Do not delete the oldest unuploaded segment simply to clear the alarm.

If retention is genuinely exhausted, preserve the last valid sequence/hash and report the affected interval as `evidence_gap`. Mosquitto may still deliver telemetry; that does not restore missing evidence.

16. A Gateway OS or package upgrade causes a regression

Preserve the failed image reference, Mosquitto data and configuration, certificates, UCI configuration, and relevant logs. Do not overwrite the only known-good SD card.

Restore the previously tested image and configuration on spare media, boot it with the same RAK5146, and repeat Gateway ID, region, local publish, journal source/sequence/hash checks, cloud checkpoint/segment upload, WAN outage, reboot, queue drain, server reconciliation, OTAA uplink, and safe downlink checks. A successful rollback must restore behavior, not merely start the services.

Canonical source: [deployment/technology-guides/04-mosquitto-mqtt.md](#). For any mutable production-status statement, check the dated baseline and verified live state.